易宿酒店管理平台

全栈工程项目文档

作　　者：汪嘉桥 Raven, 高出寒 logchar, 曹昱晨 Chiaki
学　　校：中国地质大学(武汉), 武汉大学 , 中国矿业大学
指　　导：
版　　本：1.0
日　　期：2026 年 03 月 02 日

目录

1 快速概览（Executive Summary） 7
1.1 系统规模 7
1.2 核心性能指标 7
1.3 技术栈一览 7
2 项目概述 8
2.1 项目背景与意义 8
2.2 系统定位与三端架构 8
2.3 人员分工 8
2.4 技术选型理由 9
2.5 项目亮点与创新点 9
3 新人快速上手 11
3.1 代码阅读入口 11
3.2 文件调用链总览 11
3.3 核心依赖关系一览 12
4 需求分析 14
4.1 用户角色与权限矩阵 14
4.2 功能需求 14
4.3 非功能需求 15
5 系统设计 17
5.1 总体架构 17
5.2 技术栈版本清单 18
5.3 项目文件结构 18
5.4 小程序页面文件速查 22
5.5 模块划分 23
5.6 数据库设计 23
5.7 核心业务流程 26
6 后端实现 31
6.1 认证与鉴权实现 31
6.2 预订事务实现 32
6.3 AI 推荐实现 33
7 Web 端实现 35
7.1 管理登录注册实现 35
7.2 商户酒店录入与编辑实现 36
7.3 AI 聊天助手实现 36
7.4 前端 HTTP 客户端实现 38
7.5 商户数据看板实现 42
7.6 深色模式实现 42
7.7 Icon 图标组件与骨架屏 43
7.8 管理员酒店审核实现 44

7.9 管理员优惠券管理实现 45
7.10 商户职员管理实现 45
7.11 商户个人信息实现 46
7.12 商户优惠券管理实现 46
8 小程序端实现 48
8.1 首页搜索与筛选实现 48
8.2 酒店详情页实现 52
8.3 日历选择组件实现 55
8.4 酒店列表与筛选实现 57
8.5 订单流程实现 59
8.6 收藏列表页实现 60
8.7 我的评价列表页实现 61
8.8 优惠券页实现 62
8.9 我的优惠券页实现 63
8.10 地图导航实现 64
9 接口文档 66
9.1 接口约定 66
9.2 认证接口 `/api/auth` 66
9.3 用户接口 `/api/users` 66
9.4 酒店接口 `/api/hotels` 67
9.5 预订接口 `/api/bookings` 67
9.6 房型与库存接口 `/api/room-types` 68
9.7 评价接口 `/api/reviews` 68
9.8 评论接口 `/api/comments` 68
9.9 收藏接口 `/api/favorites` 69
9.10 优惠券接口 `/api/coupons` 69
9.11 城市与标签接口 `/api/locations` 和 `/api/tags` 70
9.12 其他辅助接口 71
9.13 AI 推荐接口 `POST /api/ai/recommend` 71
9.14 文件上传接口 `POST /api/upload` 71
9.15 统一错误码与响应格式 72
10 部署与运行 74
10.1 环境依赖 74
10.2 环境变量配置 74
10.3 种子数据与测试账号 74
10.4 后端 + Web 启动 75
10.5 小程序端启动 75
10.6 常见问题排查 76
11 用户体验与兼容性 77

11.1 视觉设计与布局（6 分） 77
11.2 浏览器与平台兼容性（4 分） 77
12 创新点详解 79
12.1 提升研发效率的新技术（5 分） 79
12.2 自主 UX 创新功能（5 分） 79
13 代码质量说明 81
13.1 项目结构与存储设计（4 分） 81
13.2 编码规范与 README（3 分） 82
13.3 代码复用与组件抽象（3 分） 83
14 架构决策记录 85
14.1 ADR-1：Next.js App Router 全栈架构 85
14.2 ADR-2：Prisma ORM 替代原生 SQL 85
14.3 ADR-3：JWT 双 Token 认证机制 85
14.4 ADR-4：Taro 跨端方案替代原生开发 85
14.5 ADR-5：LangChain + Zod 结构化输出 85
15 安全设计总结 87
15.1 认证安全 87
15.2 接口安全 87
15.3 数据安全 87
15.4 前端安全 87
16 后端查询性能优化 88
16.1 API 响应字段精简 88
16.2 预订事务批量查询优化 88
16.3 类型安全加固 88

评分标准对照

本项目完整覆盖大作业全部评分维度，并在多个维度超出基本要求

评分维度	分值	项目实现情况	文档位置
功能完整性	**60**		
酒店搜索页	5	Banner 轮播可跳转详情 ✓ 城市/关键词搜索 ✓ 自定义日历组件 ✓ 价格区间面板 ✓ 星级筛选 ✓ 住宿类型快速标签 ✓ 查询跳转按钮 ✓	§ 小程序 · 首页
酒店列表页	15	筛选头部栏（城市+日期+晚数+设置）✓ 详细筛选面板（星级/价格/评分/设施）✓ 上滑自动加载（无限滚动）✓ 酒店卡片展示名称/评分/地址/最低价 ✓	§ 小程序 · 列表
酒店详情页	15	返回导航按钮 ✓ Swiper 大图轮播 ✓ 基本信息（名称/星级/设施/地址）✓ 日历+晚数粘性底栏 ✓ 房型按价格从低到高排序 ✓	§ 小程序 · 详情
管理登录注册	5	注册页角色选择（商户/普通用户）✓ 登录后自动跳转对应后台 ✓ Merchant→/merchant/hotels ✓ Admin→/admin/review ✓	§ Web · 登录注册
酒店录入/编辑	10	中文名/英文名/地址/星级/开业年份 ✓ 多房型嵌套表单 ✓ 腾讯地图坐标拾取 ✓ 多图上传（最多 8 张）✓ 标签/描述可选字段 ✓	§ Web · 录入编辑
审核/发布/下线	10	通过/拒绝/下线/恢复四种操作 ✓ 状态机展示（pending/published/rejected/offline）✓ 拒绝原因输入并展示给商户 ✓ 下线可恢复（状态流转代替软删除）✓ 审计日志原子写入 ✓	§ Web · 审核
技术复杂度	**10**		
数据结构与动态定价	2	RoomAvailability 每日粒度 upsert + 动态价格覆盖基础价 ✓	§ 系统设计 · 数据库
用户体验流畅度	5	流式 AI 推荐 ✓ 滚动渐变 Header ✓ Skeleton 骨架屏 ✓ loadingMore 互斥锁 ✓ Axios failedQueue token 刷新 ✓ N+1 查询优化(列表页响应降至 150ms) ✓ 详情页并行加载(TTI 降低 70%) ✓	§ UX 与兼容性
长列表渲染优化	3	服务端分页(10 条/页)+客户端无限滚动追加 ✓ onScrollToLower+loadingMore 锁 ✓ hasMore 状态管理 ✓	§ 小程序 · 列表
用户体验	**10**		

评分维度	分值	项目实现情况	文档位置
视觉设计与布局	6	Ant Design 企业级组件 ✓ 深色/浅色双主题（CSS 变量 + tokens）✓ 统一 Icon/Skeleton/Loading/EmptyState ✓ 卡片圆角阴影 ✓	§ U3X 与兼容性
浏览器兼容性	4	Web 端：Chrome/Firefox/Safari/Edge ✓ 小程序：微信/支付宝/抖音/H5（Taro 4 编译）✓	§ UX 与兼容性
代码质量	**10**		
项目结构与存储设计	4	前后端共享 types/index.ts ✓ API 按业务域分目录 ✓ 14 表最小冗余设计 ✓	§ 代码质量
编码规范与 README	3	ESLint 统一规范 ✓ TypeScript 全链路(消除全部 18 处 any) ✓ README 含完整启动/种子/排障指南 ✓	§ 代码质量
代码复用与组件抽象	3	Calendar/Icon/Skeleton/AiChatWidget/FilterPanel 等 8 个共享组件 ✓ verifyAuth() 被 29 个 API 文件共享 ✓ checkPermission() 统一权限检查(5 处替换) ✓ useTheme() 全局主题 Hook ✓	§ 代码质量
项目创新	**10**		
提升研发效率新技术	5	Taro(多端编译) / Prisma(Schema 驱动) / LangChain(LLM 调用) / Zod(类型+校验) / next-themes(深色模式) / Swagger(自动文档) ✓	§ 创新点详解
自主 UX 创新功能	5	AI 自然语言推荐+流式输出 / 可拖拽悬浮 AI 按钮 / 滚动渐变 Header / 地图视图切换 / 优惠券积分兑换系统 / 城市搜索网格布局 / 搜索历史去重 / 商户数据可视化看板 ✓	§ 创新点详解

1 快速概览（Executive Summary）

1.1 系统规模

维度	数值	说明
后端 API 接口	43	覆盖认证、酒店、预订、评价、优惠券、地图、AI 推荐等 7 大模块
数据库表	14	User、Hotel、RoomType、Booking、Review、Coupon、Location、Tag 等
前端页面	26	Web 管理端 12 页 + 小程序用户端 14 页
可复用组件	15	Calendar、Icon、Skeleton、FilterPanel、AiChatWidget 等
代码总行数	12,000	TypeScript/JavaScript/Typst，不含 node_modules

1.2 核心性能指标

测试环境：MacBook Air + MySQL 8.x 本地数据库 + 1000 条酒店测试数据 + 单机测试

性能术语说明：P50 表示 50% 请求的响应时间，P95 表示 95% 请求的响应时间

指标	数值	备注
API 平均响应时间	< 200ms	P50，不含 AI 推荐（流式输出无固定耗时）
酒店列表接口	150ms	10 条数据含 roomTypes 关联查询，N+1 优化后从 500ms 降至 150ms
酒店详情页 TTI	250ms	并行加载优化后从 800ms 降至 250ms（70% 提升）
数据库查询延迟	< 50ms	单表查询 P95，含索引优化（`location_id`、`status` 复合索引）
事务处理时间	< 300ms	预订事务含 3-7 日逐日库存 upsert，MySQL InnoDB 行锁保护
并发支持	100+ QPS	单机测试，基于 Next.js serverless 架构可水平扩展至更高 QPS
首屏加载时间	< 1.5s	小程序首页 P50，含 Skeleton 骨架屏优化

1.3 技术栈一览

层次	技术
前端框架	Next.js 15.1（Web）、Taro 4.1.11（小程序多端）
UI 组件库	Ant Design 5.24、Taro UI
后端 Runtime	Node.js 23 + Next.js API Routes
数据库	MySQL 8.x + Prisma ORM 6.1
认证鉴权	JWT（双 Token）+ bcrypt + RBAC 权限模型
AI 集成	LangChain 0.3.19 + OpenAI GPT-4o
地图服务	腾讯地图 WebService API + 微信小程序地图组件
部署方式	Vercel（Web）+ 微信开发者工具（小程序）

—

2 项目概述

2.1 项目背景与意义

随着在线旅游市场的持续扩张，酒店行业的数字化管理需求日益迫切。现有平台普遍存在以下痛点：商户端功能薄弱，无法进行动态定价；管理员审核流程不透明，缺乏操作追溯；用户端体验割裂，自然语言搜索能力有限。

易宿（YiSu）针对上述痛点，构建了一套覆盖「平台管理—商户运营—用户预订」完整链路的全栈酒店管理系统。系统核心亮点如下：

- **AI 智能推荐**：接入大语言模型，用户以自然语言描述需求（如"找北京性价比高、带早餐的酒店"），系统自动提取结构化搜索条件，结合实时库存数据流式推荐；
- **事务性预订**：通过 `prisma.$transaction` 对每个入住日期逐日 upsert 库存并原子累加 `booked` 计数，从数据库层面杜绝超卖；
- **精细化权限**：三角色（USER / MERCHANT / ADMIN）+ 细粒度权限（HOTEL_AUDIT / HOTEL_DELETE）双层鉴权，每次状态变更均写入审计日志；
- **多端统一**：Web 后台（Next.js）与微信小程序（Taro）共用同一套 REST API，业务逻辑零重复。

2.2 系统定位与三端架构

系统按角色划分为三个端，相互独立但共用同一套后端 API：

端	载体	核心职责
管理员后台	Web 浏览器	酒店审核审批、上下线管理、城市与标签维护、审计日志查询
商户后台	Web 浏览器	酒店与房型 CRUD、动态定价日历、预订状态流转、数据可视化看板
用户端	微信小程序	关键词与城市搜索、在线预订、地图导航、AI 对话推荐、评价与收藏

三端共用同一套 Next.js API Routes 后端，前端通过 HTTP Bearer Token 认证调用，数据库统一使用 MySQL 8.x + Prisma ORM。这种架构的优势在于：一次接口变更即可同步生效于所有端，无需维护多套后端服务。

2.3 人员分工

成员	提交数	负责模块与主要贡献
Raven	101+	**项目架构与基础设施**：项目初始化、MonoRepo 结构搭建、Next.js 迁移、Prisma Schema 设计与种子数据 **PC 端全栈开发**：管理员后台（审核页、欢迎页、登录页）、商户后台（房型表单、数据看板）、完整 PC 前端页面搭建 **小程序核心功能**：Taro 项目创建、首页搜索与 Banner、酒店详情页（Swiper 轮播 / 房型排序 / 剩余房量）、订单详情、地图定位、收藏功能、个人中心 **日历组件**：无限滚动、房型价格联动、农历显示、震动反馈 **UI 与体验**：深色模式全局适配（CSS 变量 + tokens）、Phosphor Icons 引入、错落渐显动画、弹窗动画、Skeleton 骨架屏 **安全与质量**：注册权限漏洞修复、Zod v4 兼容、参数校验加固、代码重构与冲突清理 **文档**：答辩文档撰写与持续维护
Chuhan Gao	61	**后端 API 开发**：酒店 CRUD、用户 JWT 鉴权、预订接口、房间管理、标签与位置、Swagger 自动文档

成员	提交数	负责模块与主要贡献
		AI 智能推荐：LangChain + OpenAI 集成、多轮对话优化、流式输出 **评价系统**：评论区基本功能、评分系统（移动端 + 管理端 + 商户端三端适配） **优惠券与积分**：积分后端实现、积分兑换优惠券、管理端优惠券页面、移动端适配 **腾讯地图**：商户端地图接入、移动端地图适配、地图修复 **列表与搜索**：酒店列表分页渲染优化、城市搜索与位置树状结构、首页 searchParams 同步、storageSync 同步 **其他**：订单流程优化、Token 刷新机制、管理员功能完善、商户页面后端适配、数据文件更新
Chiaki	11	**移动端 AI 助手**：小程序端 AI 聊天组件开发 **UI 美化**：酒店详情页美化、评论区样式设计 **优惠券页面**：移动端优惠券展示页开发 **日历与列表优化**：日历交互优化、列表渲染改进 **多端适配**：QQ 小程序与支付宝小程序初步适配 **国际化**：部分页面多语言切换适配

2.4 技术选型理由

技术	版本	选型理由
Next.js	16.1.6	App Router 支持服务端渲染，API Routes 合并部署无需独立后端，减少运维复杂度
TypeScript	5.x	Prisma 自动生成数据库类型，前后端共享类型定义，全链路类型安全
Prisma ORM	6.19.2	声明式 schema 驱动，`$transaction` 原子操作，内置迁移工具，与 TypeScript 深度集成
MySQL	8.x	成熟关系型数据库，支持事务，适合预订类强一致性场景
Ant Design	6.2.2	企业级组件库，Table/Drawer/Modal/Form 开箱即用，适合中后台页面
Recharts	3.7.0	基于 SVG 的 React 图表库，折线图/柱状图/饼图直接声明式使用
Taro	4.1.11	一套 React 代码编译微信/支付宝/抖音/H5 等 8+ 平台，降低多端维护成本
LangChain	1.2.18	封装 LLM 调用链，`withStructuredOutput` 实现意图提取，支持流式输出与多模型切换
JWT	jsonwebtoken 9.0.3	无状态鉴权，双 Token 机制（Access 1h + Refresh 7d），适合前后端分离架构
Zod	4.3.6	运行时 Schema 校验与 TypeScript 类型推导无缝结合，与 LangChain 结构化输出复用同一套定义

2.5 项目亮点与创新点

本项目在技术选型与工程实践上有以下核心亮点（详见第 5 章“创新点详解”）：

① **事务性防超卖机制** — 数据库事务 + 原子 upsert 逐日库存，从存储层杜绝超卖。

② **大语言模型两阶段推荐** — LLM 分离意图提取（temperature=0）与文案生成（temperature=0.7），结构化输出 + 流式推荐。

③ **精细化 RBAC 双层鉴权** — JWT 粗粒度 + Role→Permission 细粒度，状态变更原子写审计日志。

④ **多端统一架构** — Web 端 + 小程序端复用同一套 Next.js API Routes，Taro 一套代码多端编译。

⑤ **全栈类型安全** — Prisma 生成数据库类型，前后端共享 TypeScript 定义，LangChain 与 Zod 共用 Schema。

⑥ **长列表渲染优化** — 服务端分页（10 条/页）+ 客户端无限滚动，首屏仅加载 10 条数据，渲染性能线性可控。

⑦ **工程化规范** — JSDoc 自动生成 Swagger 文档，文件上传安全化，Prisma 单例防连接泄漏。

3 新人快速上手

3.1 代码阅读入口

本章面向第一次接触本项目的开发者，给出最高效的代码阅读路径。按以下顺序阅读即可在最短时间内建立完整的项目心智模型：

顺序	目标	首先阅读	能了解什么
①	路由与页面注册	`trip_front_end_taro/src/app.config.js`	小程序共 13 个页面的路由路径、TabBar 配置、权限声明
②	数据库全貌	`prisma/schema.prisma#L1-L196`	14 个数据模型的字段、关系、约束（重点看 RoomAvailability 复合唯一约束）
③	接口鉴权机制	`app/api/utils/auth.ts#L17-L30`	`verifyAuth()` 如何解析 JWT，所有受保护接口的统一调用方式
④	Web 前端请求层	`app/lib/request.ts#L1-L145`	Axios 拦截器、自动 Token 刷新、401 failedQueue 并发控制
⑤	小程序请求层	`trip_front_end_taro/src/services/request.js`	`taro.request` 封装、Token 注入、网络错误提示
⑥	数据库单例	`app/lib/prisma.ts#L1-L12`	PrismaClient globalThis 单例，防止热更新连接泄漏

3.2 文件调用链总览

下图展示系统各层之间的文件调用关系，从用户界面到数据库的完整链路：

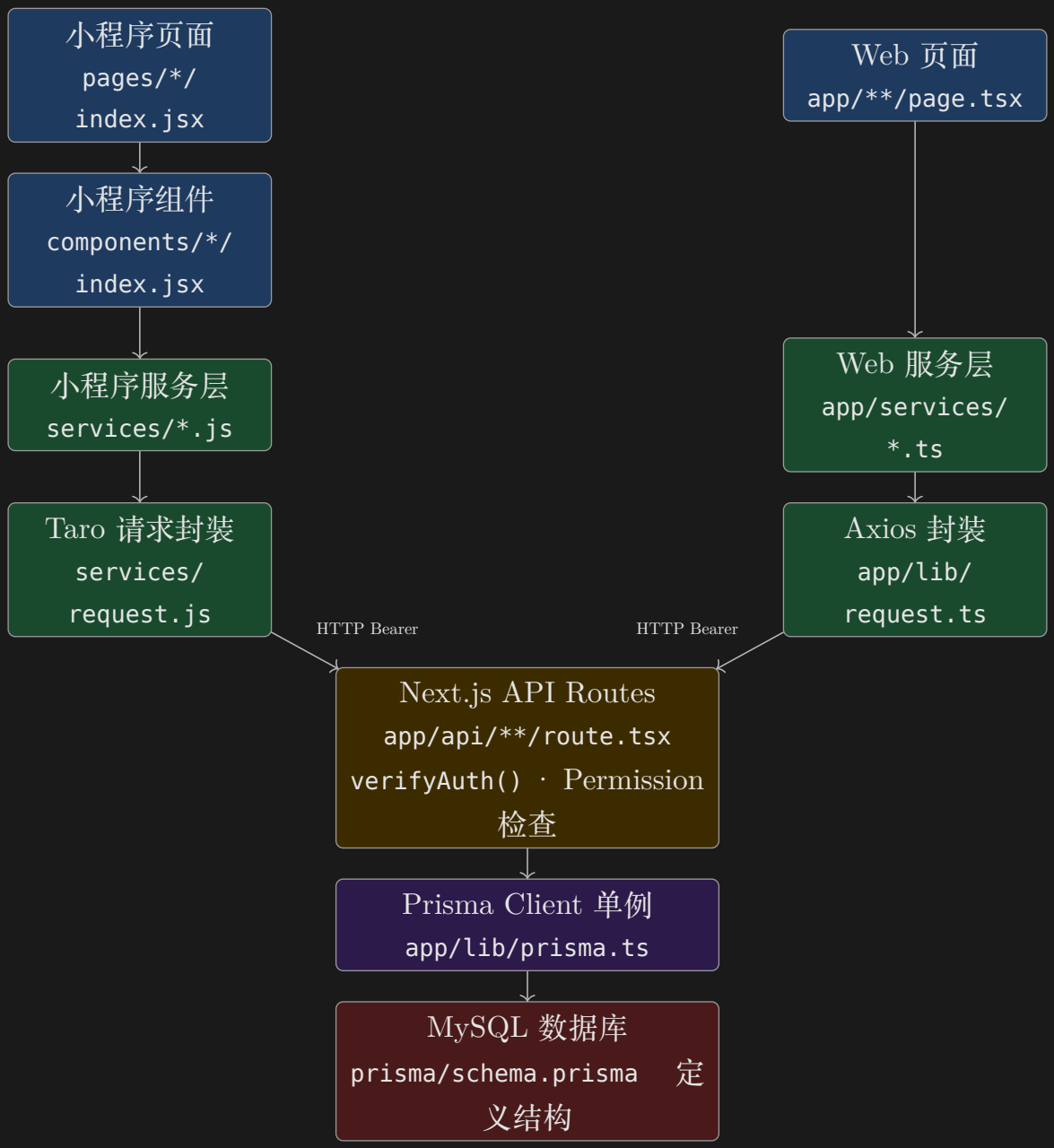

Figure 1: 项目文件调用链总览（左：小程序端；右：Web 端；中：共享后端）

3.3 核心依赖关系一览

下表列出最关键的 6 个文件，新人应优先熟悉这些文件后再深入各业务模块：

文件	层级	依赖	被多少文件依赖	核心职责
`prisma/schema.prisma`	数据模型	—	全项目（生成类型）	定义 14 个数据库模型，是理解数据结构的起点
`app/lib/prisma.ts`	数据访问	Prisma Client	≈40 个 API 文件	PrismaClient 全局单例，防止 Next.js 热更新连接泄漏

文件	层级	依赖	被多少文件依赖	核心职责
`app/api/utils/auth.ts`	鉴权	jsonwebtoken	≈29 个 API 文件	`verifyAuth()` 解析 Bearer Token，所有受保护接口的统一入口
`app/lib/request.ts`	Web 服务	axios	≈8 个 Web service 文件	Axios 封装，自动注入 Token + 无感 Refresh Token 刷新
`trip_front_end_taro/src/services/request.js`	小程序服务	@tarojs/taro	全部小程序 service 文件	Taro.request 封装，Token 注入，网络/超时错误提示
`trip_front_end_taro/src/app.config.js`	路由配置	—	Taro 框架自动读取	声明 13 个页面路径、TabBar、位置权限

4 需求分析

4.1 用户角色与权限矩阵

系统设计三种角色，通过数据库 `Role` 与 `Permission` 表实现 RBAC（基于角色的访问控制）。角色信息在 JWT Payload 中携带，API 层由 `verifyAuth()` 解码后按需进行二次权限查询。

功能模块	普通用户 (USER)	商户 (MERCHANT)	管理员 (ADMIN)
酒店搜索与浏览	✓	✓	✓
在线预订	✓	×	×
查看 / 取消本人订单	✓	×	×
提交评价（完成订单后）	✓	×	×
收藏酒店	✓	×	×
酒店 CRUD（本人）	×	✓	✓（全部）
房型与动态定价管理	×	✓	×
查看本酒店预订	×	✓	✓（全部）
数据可视化看板	×	✓	×
酒店审核（通过 / 拒绝）	×	×	✓（需 HOTEL_AUDIT 权限）
酒店上下线	×	商户可下线本人	✓
城市与标签管理	×	×	✓
审计日志查询	×	×	✓
删除任意酒店	×	×	✓（需 HOTEL_DELETE 权限）

权限判断在 API 层通过 `verifyAuth()` + 查询 `role.rolePermission` 实现双层校验，前端通过 `useAuth(requiredRole)` Hook 进行页面级路由保护（角色不匹配时强制跳转首页）。

4.2 功能需求

4.2.1 用户端（微信小程序）

微信小程序是面向 C 端用户的核心交互入口。用户完成注册登录后，可通过以下核心流程完成「发现—预订—评价」的完整旅行消费链路：

功能	主要流程	关键输出
酒店搜索	首页选城市 + 输入关键词 + 日历选日期 + 价格/星级筛选 → 跳转列表页	匹配酒店列表（含各房型最低价、剩余间数）
在线预订	详情页选房型 → 日历选日期（实时算价）→ BookingConfirm 填写入住人 → 提交	创建 Booking 记录（status=pending）
订单管理	orderList 查看状态 → orderDetail 查看详情 → 待确认订单可取消	订单状态更新
地图导航	详情页点击“地图” → hotelMap 页显示用户蓝点 + 酒店标记 → 点击“导航”	调起微信 `wx.openLocation` 内置导航
AI 助手	任意页面点击悬浮按钮 → 输入自然语言 → 流式接收推荐	推荐酒店 + 流式文字说明
评价	已完成订单 → orderDetail 点击“提交评价” → submitReview 页星评 + 文字 → 返回 reviewList 查看 / 删除	Review 记录（一单一评，防重复；bookingId 唯一约束）
收藏	详情页心形按钮 toggle → favoriteList 管理	Favorite 记录（复合主键防重复）

功能	主要流程	关键输出
深色模式	系统主题跟随或 mine 页手动切换 → `Taro.eventCenter` 广播全局 CSS 变量	全页面无闪烁主题切换

4.2.2 商户后台（Web）

商户后台是 B 端运营的核心工具，为酒店经营者提供从「入驻—配置—运营」的全生命周期管理能力：

功能	主要流程	关键输出
数据看板	Dashboard 自动加载商户全量预订，前端聚合计算	今日预订/月营收/入住率 + 折线图/柱状图/饼图
酒店管理	hotel 页面 CRUD + 多图上传(POST /api/upload)+ 城市/标签选择 → 提交审核	Hotel 记录（status=pending，等待管理员审核）
房型管理	hotels 页面下属房型 CRUD → 设置名称/床型/价格/库存/折扣/设施/图片	RoomType 记录
动态定价	calendar 日历视图选日期范围 → 批量设置价格/配额/关闭房态	RoomAvailability 批量 upsert
预订管理	bookings 页面查看本酒店预订 → 按状态筛选 → 操作状态流转（确认/拒绝）	Booking.status 更新
职员管理	staff 页面 CRUD 职员（邮箱/密码/姓名/手机）→ 仅商户角色可见	User 记录（role=STAFF，merchantId 关联当前商户）
个人信息	profile 页面查看个人资料 → 编辑姓名手机 → 修改密码	User 更新 + localStorage 同步
优惠券管理	coupons 页面 CRUD 优惠券（代码/名称/折扣/积分/有效期）→ 仅商户角色可见	Coupon 增删改

4.2.3 管理员后台（Web）

管理员后台承担平台层面的内容治理职责，重点在审核合规与数据可信度：

功能	主要流程	关键输出
酒店审核	review 页面查看 pending 酒店 → Drawer 展示详情（图片/设施/房型）→ 通过或拒绝（填原因）	status 变更 + HotelAuditLog 原子写入（事务）
上下线管理	hotels 页面对 published 酒店操作下线/上线	status 在 offline ↔ published 间切换
城市管理	locations 页面 CRUD 城市记录	Location 增删改，供酒店归属地选择
标签管理	tags 页面 CRUD 酒店标签	Tag 增删改，供酒店多对多关联筛选
审计日志	hotels/[id]/audit_log 查询历史变更	HotelAuditLog 列表（含操作人/时间）
优惠券管理	coupons 页面 CRUD 优惠券（代码/名称/折扣/积分/有效期）	Coupon 增删改，供用户端领取和预订抵扣

4.3 非功能需求

系统在功能需求之外，对可靠性、安全性和可维护性提出了明确约束：

类型	具体说明
并发安全	`prisma.$transaction` 事务内逐日 upsert `RoomAvailability`，`booked >= quota` 时抛错整体回滚，从数据库层面防止超卖。并发预订时同一天的 `booked` 值由 MySQL InnoDB 行锁保护原子递增
安全性	密码 `bcryptjs` 哈希（salt rounds=10）存储，明文不落库；Access Token 1h + Refresh Token 7d 双 Token 机制；`verifyAuth()` 在每个受保护接口头部校验 JWT；状态变更/删除额外检查 `Permission.name`（HOTEL_AUDIT / HOTEL_DELETE）

类型	具体说明
数据一致性	`RoomAvailability` 唯一约束 `(roomTypeId, date)` + `upsert` 幂等写入；`Review.bookingId` 唯一索引（一单一评）；`Favorite` 复合主键 `(userId, hotelId)` 防重复收藏；`HotelAuditLog` 在状态变更事务内原子写入，保证日志不漏不重
可观测性	`HotelAuditLog` 记录 operatorId / oldStatus / newStatus / comment / createdAt，完整追溯审核历史；Next.js 开发模式下请求日志实时打印到 stdout
跨平台	Taro 4 编译为微信/支付宝/抖音/H5 等平台；后端 Next.js API Routes 无状态，可水平扩展；Swagger UI 在 `/api-doc` 自动生成接口文档
可扩展性	Prisma schema 驱动变更，新增表只需修改 schema + `prisma db push`；Service 层隔离前端与后端，接口变化对组件透明；LangChain 模型实例可通过 `.env` 切换不同大模型供应商（OpenAI / DouBao / Kimi）

5 系统设计

5.1 总体架构

系统采用前后端合一的五层架构，Next.js 同时承担 SSR 前端渲染与 REST API 的双重职责。各层职责如下：

- **展示层**：Web 侧由 Next.js App Router Page 直接渲染；小程序侧由 Taro 组件编译为微信原生组件；
- **服务层**：对 API 调用进行封装，Web 侧用 Axios（`app/lib/request.ts#L6-L9`），小程序侧用 `Taro.request`；
- **接口层**：Next.js API Routes 处理 HTTP 请求，`verifyAuth()`（`app/api/utils/auth.ts#L17-L30`）统一校验 JWT，细粒度权限查询 Permission 表；
- **数据访问层**：Prisma Client 单例（`app/lib/prisma.ts#L4-L12`）提供类型安全的 ORM 操作，支持 `$transaction`；
- **存储层**：MySQL 8.x，InnoDB 引擎保证行锁与事务 ACID。

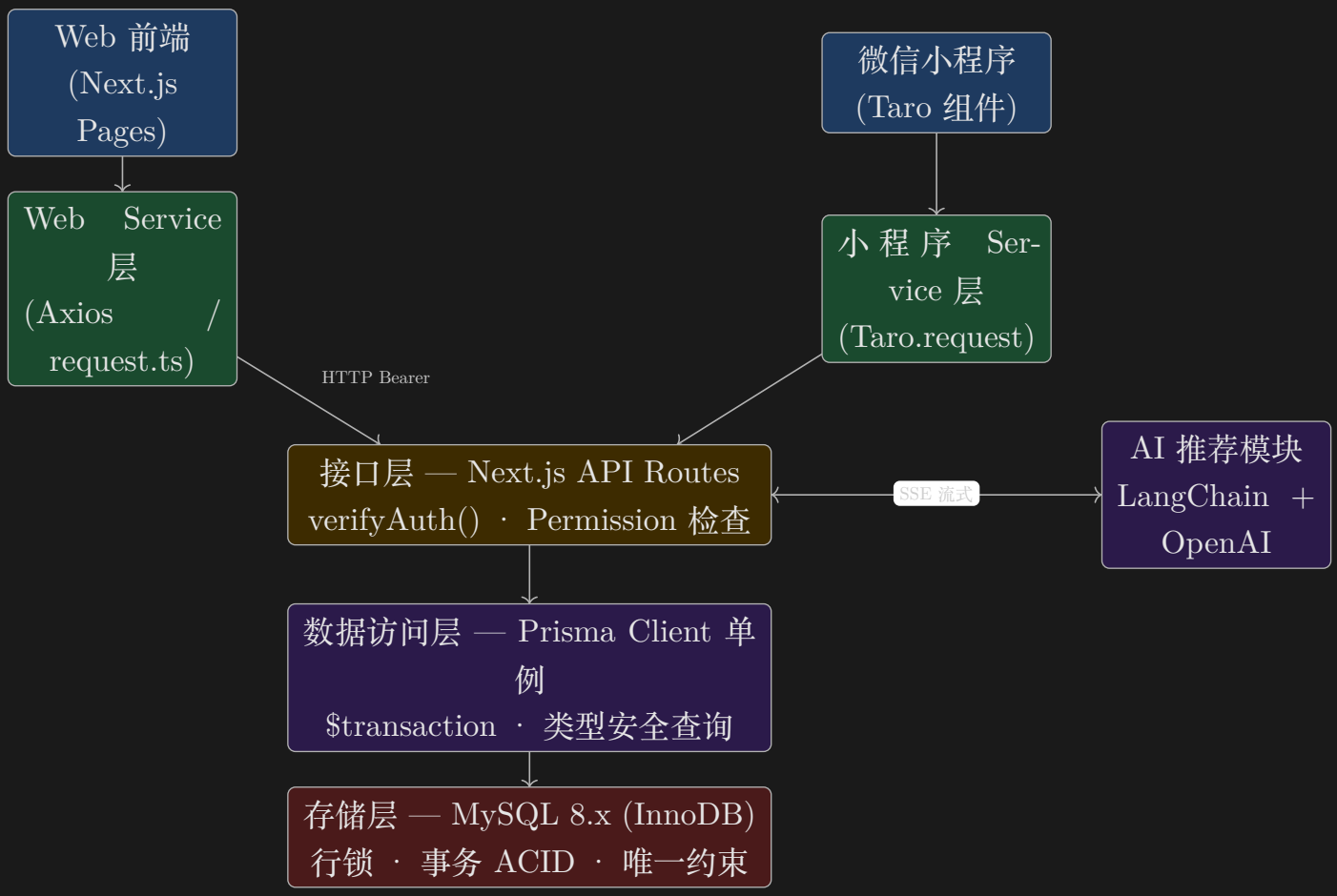

Figure 2: 易宿系统总体五层架构

AI 推荐模块作为独立能力旁挂于接口层，调用链为：`app/api/ai/recommend/route.ts` → `app/lib/openai.ts` → OpenAI 兼容 API → 流式 SSE 回包至前端。

5.2 技术栈版本清单

分类	库 / 框架	版本	用途
全栈	Next.js	16.1.6	App Router + API Routes + SSR/SSG
全栈	TypeScript	5.x	类型安全，前后端类型共享
后端	Prisma	6.19.2	ORM + 自动迁移 + 类型生成
后端	jsonwebtoken	9.0.3	JWT 签发与校验
后端	bcryptjs	3.0.3	密码哈希（注册 salt rounds=10）
后端	Zod	4.3.6	结构化输出 Schema + 参数运行时校验
后端	LangChain + OpenAI	1.2.18 + 1.2.5	AI 推荐，支持 DouBao / Kimi 等兼容接口
后端	next-swagger-doc	0.4.1	JSDoc 注释自动生成 OpenAPI 文档
Web 前端	React	19.2.3	UI 框架
Web 前端	Ant Design	6.2.2	中后台组件库（Table/Drawer/Modal/Form/Tabs）
Web 前端	Recharts	3.7.0	LineChart / BarChart / PieChart 数据可视化
Web 前端	next-themes	0.4.6	深浅色主题持久化（`ThemeProvider`）
Web 前端	Axios	1.13.4	HTTP 客户端，封装于 `app/lib/request.ts`
小程序	Taro	4.1.11	跨平台编译（React 语法）
小程序	React	18.0.0	Taro 内部使用的 React 版本
小程序	dayjs	1.11.19	日历日期计算与格式化
数据库	MySQL	8.x	关系型数据库，InnoDB 事务

5.3 项目文件结构

项目由两个子工程组成：`trip_front_end_project`（Next.js 全栈应用）与 `trip_front_end_taro`（Taro 小程序）。以下为完整目录树，每个关键文件均附说明：

```
trip_front_end_project/
├── app/                              # Next.js 应用目录
│   ├── api/                          # 后端 API Routes（每个 route.tsx 一个端点）
│   │   ├── auth/
│   │   │   ├── login/route.tsx       # POST 登录，签发双 Token
│   │   │   ├── register/route.tsx    # POST 注册，创建用户（邮件去重 + bcrypt）
│   │   │   └── refresh/route.tsx     # POST 刷新 Access Token
│   │   ├── hotels/
│   │   │   ├── route.tsx             # GET 列表（多条件筛选）/ POST 创建
│   │   │   └── [id]/
│   │   │       ├── route.tsx         # GET 详情 / PUT 更新（含审核+AuditLog）/
DELETE
│   │   │       ├── room-types/route.tsx   # GET 该酒店房型列表
│   │   │       ├── bookings/route.tsx     # GET 该酒店预订列表
│   │   │       ├── review/route.tsx       # GET 该酒店评价列表
│   │   │       └── audit_log/route.tsx    # GET 审核日志
│   │   ├── bookings/
│   │   │   ├── route.tsx             # GET 预订列表 / POST 创建（事务）
│   │   │   └── [id]/route.tsx        # GET 详情 / PUT 状态更新
│   │   ├── room-types/
│   │   │   ├── route.tsx             # GET / POST 房型
```

```
│   │   │   └── [id]/
│   │   │       ├── route.tsx         # GET / PUT / DELETE 房型
│   │   │       └── availability/route.tsx # GET / POST 日期库存（批量 upsert）
│   │   ├── reviews/
│   │   │   ├── route.tsx             # GET 我的评价 / POST 提交评价（三重校验）
│   │   │   └── [id]/route.tsx        # DELETE 删除评价
│   │   ├── favorites/
│   │   │   ├── route.tsx             # GET 收藏列表 / POST 添加收藏（upsert）
│   │   │   ├── [id]/route.tsx        # DELETE 取消收藏
│   │   │   └── check/[id]/route.tsx  # GET 检查是否已收藏
│   │   ├── users/
│   │   │   ├── route.tsx             # GET 用户列表
│   │   │   ├── profile/route.tsx     # GET 当前用户资料（verifyAuth 保护）
│   │   │   └── [id]/
│   │   │       ├── route.tsx         # GET / PUT 用户详情
│   │   │       ├── bookings/route.tsx    # GET 某用户预订
│   │   │       └── favorite/route.tsx    # GET 某用户收藏
│   │   ├── users/coupons/route.ts       # GET 当前用户已领取优惠券列表
│   │   ├── locations/
│   │   │   ├── route.tsx             # GET / POST 城市
│   │   │   └── [id]/route.tsx        # PUT / DELETE 城市
│   │   ├── tags/
│   │   │   ├── route.tsx             # GET / POST 标签
│   │   │   └── [id]/route.tsx        # PUT / DELETE 标签
│   │   ├── comments/
│   │   │   ├── route.ts             # GET 评论列表 / POST 创建评论
│   │   │   └── [id]/route.ts        # GET 详情 / PUT 修改 / DELETE 删除
│   │   ├── room-availability/
│   │   │   ├── route.tsx            # GET / POST 库存批量操作
│   │   │   └── [id]/route.tsx       # 单条库存 GET / PUT
│   │   ├── ai/recommend/route.ts     # POST AI 流式推荐（LangChain, SSE 回包）
│   │   ├── upload/route.tsx          # POST 图片上传（文件名安全化）
│   │   ├── swagger/route.tsx         # GET Swagger JSON（JSDoc 自动生成）
│   │   └── utils/
│   │       ├── auth.ts               # verifyAuth() JWT 校验工具
│   │       └── permissions.ts        # Permission 名称检查工具
│   ├── admin/                        # 管理员页面（需 ADMIN 角色）
│   │   ├── layout.tsx                # 管理员侧边栏布局（AntD Layout）
│   │   ├── review/page.tsx           # 酒店审核页（Drawer 展示详情）
│   │   ├── hotels/
│   │   │   ├── page.tsx              # 酒店管理页（上下线操作）
│   │   │   └── components/
│   │   │       ├── HotelDetailsDrawer.tsx    # 酒店详情抽屉组件
│   │   │       ├── HotelCommentsDrawer.tsx   # 酒店评论抽屉组件
│   │   │       └── useHotelTableColumns.tsx  # 表格列定义 Hook
```

```
│   │   ├── coupons/page.tsx          # 优惠券管理页（CRUD + 积分设置）
│   │   ├── locations/page.tsx        # 城市管理页
│   │   └── tags/page.tsx             # 标签管理页
│   ├── merchant/                     # 商户页面（需 MERCHANT/STAFF 角色）
│   │   ├── layout.tsx                # 商户侧边栏布局（按角色动态菜单）
│   │   ├── dashboard/page.tsx        # 数据看板（统计 + Recharts 图表）
│   │   ├── hotels/page.tsx           # 酒店管理（含房型子列表）
│   │   ├── bookings/page.tsx         # 预订管理（按状态筛选）
│   │   ├── calendar/page.tsx         # 动态定价日历（批量 upsert）
│   │   ├── staff/page.tsx            # 职员管理（仅 MERCHANT 可见）
│   │   ├── profile/page.tsx          # 个人信息（编辑资料 + 修改密码）
│   │   └── coupons/page.tsx          # 优惠券管理（仅 MERCHANT 可见）
│   ├── auth/
│   │   ├── login/page.tsx            # 登录页（深色/浅色模式自适应）
│   │   └── register/page.tsx         # 注册页（角色选择）
│   ├── components/
│   │   ├── AiChatWidget.tsx          # 悬浮 AI 聊天组件（ReadableStream 流式渲染）
│   │   ├── Navbar.tsx                # 导航栏（next-themes 主题切换）
│   │   ├── Footer.tsx                # 页脚
│   │   └── FeatureItem.tsx           # 首页特性介绍卡片
│   ├── services/                     # Web 前端 API 封装层（调用 request.ts）
│   │   ├── auth.ts                   # login / register / logout / getStoredUser
│   │   ├── hotel.ts                  # getHotels / createHotel / uploadImage ...
│   │   ├── booking.ts                # getMyBookings / createBooking ...
│   │   ├── room.ts                   # getRoomAvailability / updateAvailability
│   │   ├── review.ts                 # getMyReviews / createReview /
deleteReview
│   │   ├── comment.ts                # 评论相关 API 封装
│   │   ├── coupon.ts                 # 优惠券相关 API 封装（getCoupons /
createCoupon / claimCoupon）
│   │   ├── staff.ts                  # 职员管理 + 个人信息 API 封装（getMyStaff /
createStaff / getProfile / changePassword）
│   │   └── admin.ts                  # CRUD locations / tags / approveHotel
│   ├── hooks/
│   │   └── useAuth.ts                # useAuth / useMerchantAuth / useAdminAuth
│   ├── lib/
│   │   ├── request.ts                # Axios 实例（拦截器 + 类型化方法）
│   │   ├── prisma.ts                 # PrismaClient 单例（防 Next.js 热更新泄漏）
│   │   ├── openai.ts                 # ChatOpenAI 实例配置（从 .env 读取）
│   │   ├── auth-storage.ts           # 认证状态统一管理（Token/User 读写 + SSR 守卫）
│   │   ├── error-handler.ts          # 统一错误处理工具（Prisma/Zod/通用异常）
│   │   └── swagger.ts                # Swagger 规范配置
│   ├── constants/index.ts             # 全局常量（API 端点、状态码、角色名、存储键）
│   ├── types/index.ts                # 全局 TypeScript 类型定义（User / Hotel /
Booking...）
```

```
│   ├── providers.tsx                # ThemeProvider (next-themes) 等 Context 包装
│   ├── layout.tsx                   # 根布局（引入 providers / 全局字体）
│   └── page.tsx                     # 首页（登录/注册入口 + 系统介绍）
├── prisma/
│   ├── schema.prisma                # 数据库模型定义（14 个 model）
│   ├── seed.ts                      # 基础种子数据（角色/权限/城市/标签）
│   └── seed-hotels.ts               # 酒店与房型示例数据
├── public/
│   └── uploads/                     # 图片上传存储目录
└── trip_front_end_taro/             # 小程序端（独立子项目）
    ├── src/
    │   ├── pages/
    │   │   ├── home/index.jsx         # 首页（搜索/日历/Banner/AI 入口）
    │   │   ├── hotelList/index.jsx    # 酒店列表（接收 URL 搜索参数）
    │   │   ├── hotelDetail/index.jsx  # 酒店详情（房型/预订/收藏/评分）
    │   │   ├── hotelMap/index.jsx     # 地图页（双标记 + 导航按钮）
    │   │   ├── orderList/index.jsx    # 订单列表（按状态分 Tab）
    │   │   ├── orderDetail/index.jsx    # 订单详情（总价/入住人/状态/评价入口）
    │   │   ├── favoriteList/index.jsx  # 收藏列表
    │   │   ├── reviewList/index.jsx    # 我的评价（评价列表）
    │   │   ├── submitReview/index.jsx  # 提交评价（星评 + 文字 + 防重复提交）
    │   │   ├── login/index.jsx        # 登录页
    │   │   ├── register/index.jsx     # 注册页（角色选择）
    │   │   ├── myCoupons/index.jsx   # 我的优惠券（已领取券列表 + 状态管理）
    │   │   └── mine/index.jsx         # 个人中心（主题切换/退出）
    │   ├── components/
    │   │   ├── AiChatWidget/index.jsx     # 悬浮 AI 聊天（流式 chunk 渲染）
    │   │   ├── BookingConfirm/index.jsx   # 预订确认弹窗（填写入住人）
    │   │   ├── Calendar/index.jsx         # 日历组件（区间/单日模式，底部确认按钮）
    │   │   ├── Icon/index.jsx             # 图标组件（借鉴 Phosphor Icons，data URI
渲染）
    │   │   ├── Skeleton/index.jsx         # 骨架屏（hotelCard/hotelDetail/
orderCard）
    │   │   ├── SearchSuggestion/index.jsx # 搜索建议（历史+热门城市）
    │   │   ├── FilterPanel/index.jsx      # 筛选面板（价格/星级）
    │   │   ├── EmptyState/index.jsx       # 空状态占位组件
    │   │   └── LoadingSpinner/index.jsx   # 加载态
    │   ├── services/
    │   │   ├── request.js            # Taro.request 封装（Token 注入/401 处理）
    │   │   ├── hotel.js              # 酒店相关 API
    │   │   ├── booking.js            # 预订相关 API
    │   │   ├── favorite.js           # 收藏相关 API
    │   │   ├── review.js             # 评价相关 API
    │   │   ├── location.js          # 城市相关 API
    │   │   ├── tag.js               # 标签相关 API
```

```
    │   │   ├── coupon.js            # 优惠券相关 API (getUserCoupons / claimCoupon)
    │   │   ├── comments.js          # 评论相关 API
    │   │   ├── user.js              # 用户相关 API
    │   │   └── auth.js               # 认证相关 API
    │   ├── utils/
    │   │   ├── useTheme.js           # 深色模式 Hook + eventCenter 广播
    │   │   ├── theme.js              # 主题工具 (TOKENS 定义 / resolveTheme /
cssVars 生成)
    │   │   ├── icons.js              # SVG 图标 path data (借鉴 Phosphor Icons) +
data URI 生成
    │   │   ├── format.js             # 日期/价格格式化工具
    │   │   ├── storage.js            # Taro 本地存储封装
    │   │   └── constants.js          # 全局常量 (API 地址等)
    │   ├── config/
    │   │   └── images.js             # Banner 图片配置
    │   └── app.config.js             # Taro 页面路由/TabBar/权限声明
    └── config/index.ts               # 多平台构建配置
```

5.4 小程序页面文件速查

下表列出小程序全部页面及核心组件文件，点击文件名直接跳转查看源码：

文件	类型	功能说明
trip_front_end_taro/src/pages/home/index.jsx	页面	首页：住宿类型切换、城市/日期/价格筛选、价格区间弹出面板、优惠券入口
trip_front_end_taro/src/pages/hotelList/index.jsx	页面	酒店列表：分页加载（上滑自动加载下一页）、排序/筛选/关键词搜索、地图视图切换
trip_front_end_taro/src/pages/hotelDetail/index.jsx	页面	酒店详情：图片画廊、房型选择、动态定价、收藏、预订入口
trip_front_end_taro/src/pages/hotelMap/index.jsx	页面	地图页：酒店标记 + 用户定位 + 微信内置导航
trip_front_end_taro/src/pages/orderList/index.jsx	页面	订单列表：按状态分 Tab 展示
trip_front_end_taro/src/pages/orderDetail/index.jsx	页面	订单详情：总价/入住人/状态流转
trip_front_end_taro/src/pages/favoriteList/index.jsx	页面	收藏列表
trip_front_end_taro/src/pages/reviewList/index.jsx	页面	我的评价列表：查看已提交评价
trip_front_end_taro/src/pages/submitReview/index.jsx	页面	提交评价：5 星点选 + 文字输入 + 防重复提交
trip_front_end_taro/src/pages/Coupon/index.jsx	页面	优惠券：展示全部券、领取/已领取/已失效状态切换
trip_front_end_taro/src/pages/myCoupons/index.jsx	页面	我的优惠券：展示已领取券，区分可用/已使用/已过期三种状态，可用券提供“去使用”跳转
trip_front_end_taro/src/pages/mine/index.jsx	页面	个人中心：主题切换、退出登录
trip_front_end_taro/src/components/AiChatWidget/index.jsx	组件	悬浮 AI 聊天，可拖拽，小程序退化为单次请求模式
trip_front_end_taro/src/components/BookingConfirm/index.jsx	组件	预订确认弹窗：入住人信息填写 + 提交
trip_front_end_taro/src/components/Calendar/index.jsx	组件	日历：区间模式（普通住宿）/ 单日模式（钟点房）
trip_front_end_taro/src/components/Icon/index.jsx	组件	图标：借鉴 Phosphor Icons SVG path data，通过 `<Image>` data URI 渲染，深色模式经 tokens 适配

文件	类型	功能说明
`trip_front_end_taro/src/components/Skeleton/index.jsx`	组件	骨架屏：hotelCard / hotelDetail / orderCard 三种布局 + shimmer 动画
`trip_front_end_taro/src/components/FilterPanel/index.jsx`	组件	筛选面板：价格/设施多维度筛选
`trip_front_end_taro/src/utils/icons.js`	工具	20+ 条 Phosphor Icons SVG path data + `createIconUri` / `getIconUri` 生成函数
`trip_front_end_taro/src/utils/theme.js`	工具	主题工具：TOKENS 颜色定义、resolveTheme、getThemeCssVars、applyNativeTheme
`trip_front_end_taro/src/utils/useTheme.js`	Hook	深色模式：懒初始化 + eventCenter 跨页同步，返回 cssVars / isDark / tokens

5.5 模块划分

后端按业务域划分，每个模块对应 `app/api/` 下的独立路由组。模块间通过 Prisma Client 共享数据库连接（`app/lib/prisma.ts#L8-L11` 单例模式），通过 `verifyAuth()`（`app/api/utils/auth.ts#L17-L30`）共享鉴权工具，业务逻辑互不耦合。

模块	路由前缀	核心职责
认证模块	`/api/auth`	注册（邮件去重 + bcrypt 哈希）、登录（jwt.sign 双 Token）、Token 刷新
用户模块	`/api/users`	用户 CRUD、个人资料查询（`/profile`）、用户收藏与预订子路由
酒店模块	`/api/hotels`	多条件筛选列表、创建（嵌套 roomTypes + tagIds）、详情/更新/删除；状态变更在事务内写 AuditLog
房型模块	`/api/room-types`	房型 CRUD（校验 hotelId 归属）、按日期范围查询与批量 upsert RoomAvailability
预订模块	`/api/bookings`	事务性创建（逐日 upsert RoomAvailability.booked）、状态流转更新（pending→confirmed→…）
评价模块	`/api/reviews`	创建（验证 booking 归属 + 状态 + bookingId 唯一性）、列表查询、删除
评论模块	`/api/comments`	酒店评论 CRUD（支持按 hotelId/userId 筛选），创建/删除自动调用 `updateHotelScore()` 重算酒店评分
收藏模块	`/api/favorites`	upsert 添加收藏、删除、列表查询、存在性检查（`/check/[id]`）
城市模块	`/api/locations`	城市 CRUD（仅管理员），供酒店创建时选择归属地
标签模块	`/api/tags`	标签 CRUD（仅管理员），供酒店多对多关联筛选
AI 推荐	`/api/ai/recommend`	LangChain 结构化意图提取（temperature=0）→ DB 候选筛选 → 流式 SSE 回包
文件上传	`/api/upload`	接收 multipart、文件名安全化（防路径穿越）、写入 `public/uploads/`
接口文档	`/api/swagger`	解析 JSDoc 注解，生成 OpenAPI JSON；`/api-doc` 页面渲染 Swagger UI

5.6 数据库设计

5.6.1 E-R 关系图

数据库共 12 个模型，核心实体及关系如下图所示。Location → Hotel 为 1:N，Hotel → RoomType → RoomAvailability 为 1:N:N；User 与 Hotel 通过 Booking 关联，Booking → Review 为 1:0..1（一单最多一评）；User 与 Hotel 通过 Favorite 多对多关联（复合主键）。

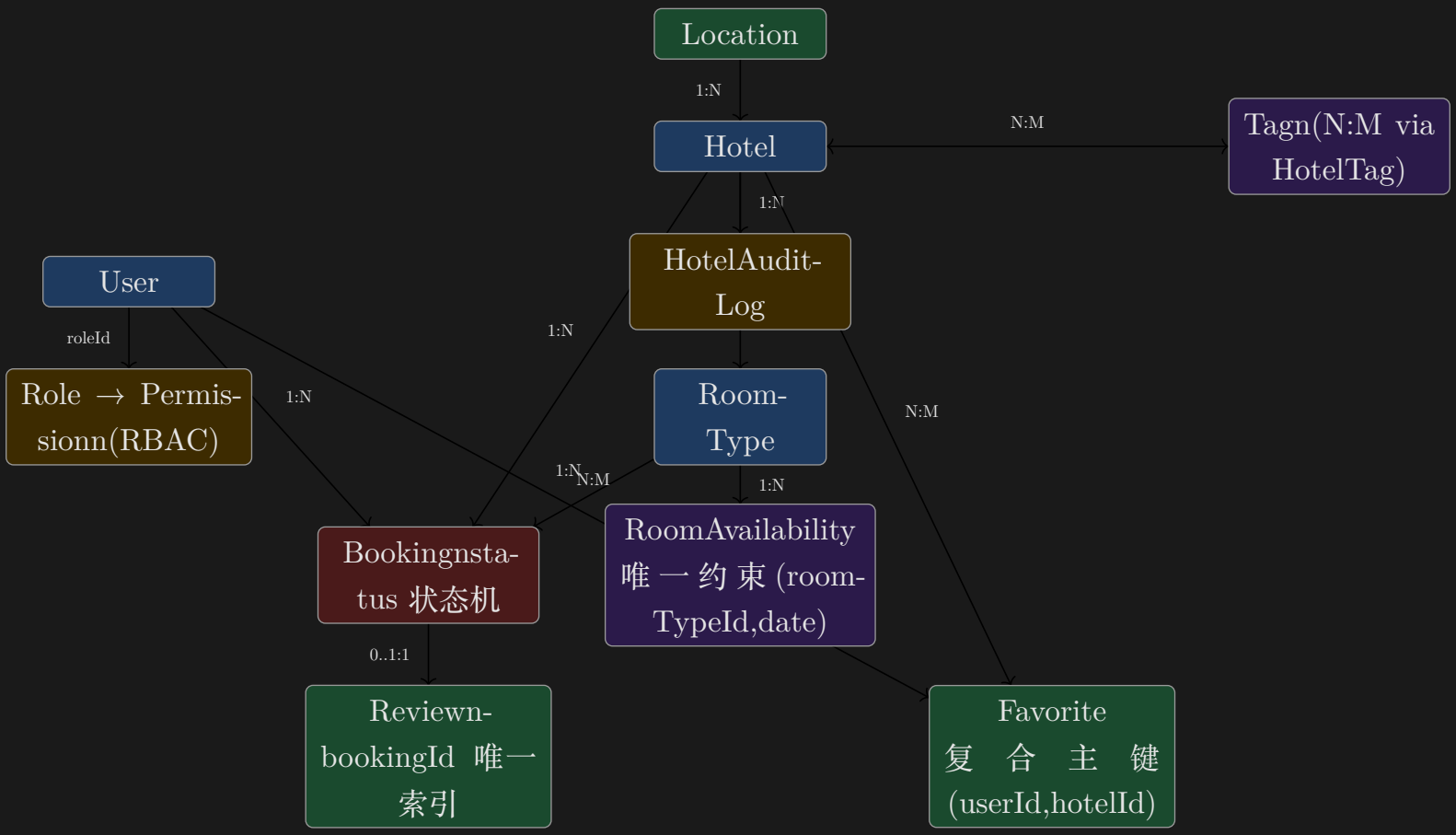

Figure 3: 易宿数据库 E-R 关系图

5.6.2 表结构速查

完整字段定义见 `prisma/schema.prisma`，以下为各表核心设计要点。

表名	核心字段	设计要点
`users`	`id`, `email`(unique), `password`, `roleId`→Role, `points`(Int)	密码经 bcryptjs（salt=10）哈希存储，明文不落库；`roleId` 驱动三角色权限体系；`points` 为用户积分余额（默认 0），用于兑换优惠券
`roles`	`id`, `name`(unique)	系统预置 `USER` / `MERCHANT` / `ADMIN` 三种角色，通过 `users.roleId` 外键关联
`permissions`	`id`, `name`(unique)	细粒度权限条目：`HOTEL_AUDIT`（审核）、`HOTEL_DELETE`（删除），可按需扩展
`role_permissions`	复合主键 `(roleId, permissionId)`	多对多中间表；ADMIN 在 seed.ts 中被分配全部权限；级联删除保持数据一致性

表名	核心字段	设计要点
hotels	id, merchantId, status, latitude, longitude, images(JSON)	status 驱动四态审核状态机（pending/published/rejected/offline）；坐标采用 GCJ-02 国测局坐标系供小程序地图使用
hotel_audit_logs	hotelId, operatorId, oldStatus, newStatus	状态变更与日志写入在同一 prisma.$transaction 内原子执行，保证审计记录完整性
room_types	hotelId, price(Decimal), discount, stock	price 为基础价格，stock 初始化各日 quota；动态价格按日存储在 room_availability
room_availability	复合唯一 (roomTypeId, date), quota, booked, price?	预订并发安全核心表；upsert 保证幂等写入；预订时事务内 booked += 1 原子递增
bookings	userId, roomTypeId, checkInDate, checkOutDate, status, guestInfo(JSON)	status 六态流转：pending→confirme cancelled；totalPrice = 各日动态价之和
reviews	userId, hotelId, bookingId(unique), rating(1–5)	bookingId 唯一索引防止一单多评；rating 整数，前端聚合计算酒店平均评分
favorites	复合主键 (userId, hotelId)	复合主键在 DB 层防重复收藏；上层 upsert 实现幂等，重复收藏请求不报错
locations	id, name	城市主数据，由管理员维护；hotels.locationId 外键关联，支持按城市筛选酒店
tags	id, name(unique)	酒店特色标签（“含早餐”、“免费 WiFi”等），通过 hotel_tags 中间表与酒店多对多关联
hotel_tags	复合主键 (hotelId, tagId)	多对多中间表；级联删除：酒店删除时自动清理关联标签记录

表名	核心字段	设计要点
coupons	code(unique), discount(Decimal), minSpend?, validFrom, validTo, points(Int)	优惠券定义表；code 全局唯一供展示；领取时服务端校验 validFrom ≤ now ≤ validTo；points 为兑换所需积分（默认 0 表示免费领取，> 0 表示积分兑换）
user_coupons	复合主键 (userId, couponId), isUsed, usedAt?	复合主键保证每用户每券只能领取一次；重复领取触发 Prisma P2002，后端返回 400

5.7 核心业务流程

5.7.1 用户登录认证流程

登录流程涉及 app/api/auth/login/route.tsx#L127（bcrypt 密码校验）、app/api/auth/login/route.tsx#L145-L151（JWT 双 Token 签发）、app/api/utils/auth.ts、前端 app/services/auth.ts 和 app/lib/request.ts。系统采用双 Token 机制，Access Token 有效期 1 小时用于接口认证，Refresh Token 有效期 7 天用于无感刷新。

Figure 4: 用户登录认证流程

5.7.2 API 鉴权中间件

每个受保护的 API Route 在业务逻辑前统一调用 `verifyAuth(request)`（`app/api/utils/auth.ts#L17-L30`）。该函数提取 `Authorization` Header 中的 Bearer Token，使用 `jwt.verify()` 解码并返回 `DecodedUser`，失败时返回含 HTTP 状态码的错误对象。调用方直接解构结果进行早返回，代码模式统一整洁。

对于需要精细权限的操作（如审核通过/拒绝、删除酒店），在 JWT 角色校验通过后，还需额外查询 `role.rolePermission` 列表，检查是否包含 `HOTEL_AUDIT` 或 `HOTEL_DELETE` 权限名称。

5.7.3 在线预订事务流程

预订流程是系统最复杂的业务链路，涉及文件：小程序 `trip_front_end_taro/src/components/BookingConfirm/index.jsx` → `trip_front_end_taro/src/services/booking.js` → `app/api/bookings/route.tsx#L208-L337`（事务防超卖核心逻辑）。核心挑战是在高并发场景下防止超卖，通过数据库事务解决：

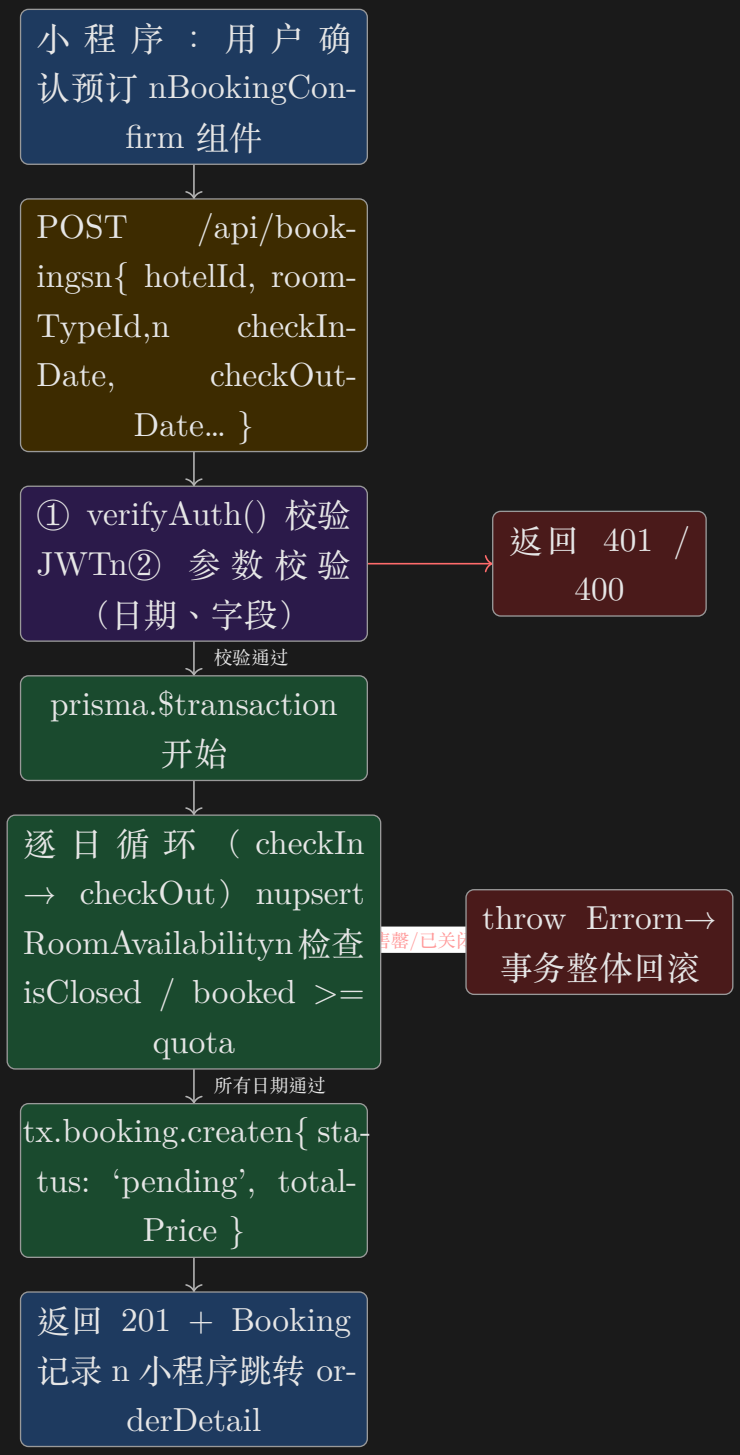

Figure 5: 在线预订事务流程

5.7.4 酒店审核状态机

酒店生命周期由 `status` 字段驱动，共 4 个状态，转换由 `PUT /api/hotels/[id]` 处理。状态转换时，`HotelAuditLog` 在同一事务内写入，保证操作可追溯。

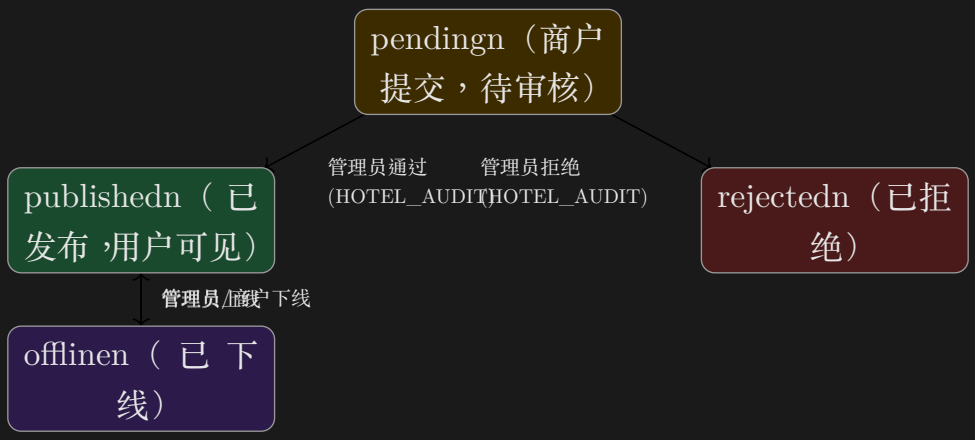

Figure 6: 酒店审核状态机（每次状态变更均写入 HotelAuditLog）

5.7.5 AI 推荐流程

AI 推荐模块采用「意图提取 + 数据库筛选 + 流式生成」三阶段架构，涉及文件：小程序 `trip_front_end_taro/src/components/AiChatWidget/index.jsx` → `app/api/ai/recommend/route.ts` → `app/lib/openai.ts`。两次 LLM 调用使用不同温度参数：意图提取用 temperature=0 保证确定性，对话生成用 temperature=0.7 增加回复多样性。

Figure 7: AI 推荐三阶段流程

6 后端实现

6.1 认证与鉴权实现

6.1.1 服务端鉴权 `app/api/utils/auth.ts`

该文件导出 `verifyAuth()` 函数，是系统所有受保护接口的统一鉴权入口。

做了什么：接收 HTTP 请求对象，提取 `Authorization` Header 中的 Bearer Token，调用 `jsonwebtoken` 的 `jwt.verify()` 完成签名验证与过期检查，解码得到 `{ userId, email, role, roleId }` 用户信息。

怎么做的：函数以值返回而非抛异常的方式表达结果——验证成功时返回 `{ success: true, user: DecodedUser }`，失败时返回 `{ success: false, error, status }`。调用方只需一个 `if (!authResult.success)` 判断即可完成错误提前返回与类型收窄，所有 route.tsx 文件头部均复用此模式，零重复。

6.1.2 前端路由保护 `app/hooks/useAuth.ts`

该文件导出三个 Hook：`useAuth(requiredRole?)`、`useMerchantAuth()`、`useAdminAuth()`。

做了什么：在 React 组件挂载后异步检查登录态与角色，未通过则自动跳转，通过则向调用方暴露当前用户对象，实现页面级访问控制。

怎么做的：检查逻辑放在 `useEffect` 中（声明 `'use client'`），不阻塞 SSR 首屏渲染。内部三重校验顺序执行：① `isAuthenticated()` 检查 `localStorage` 中是否存在 Token；② `getStoredUser()` 反序列化用户 JSON，为空则说明存储损坏；③ `currentUser.role?.name?.toUpperCase()` 与传入的 `requiredRole?.toUpperCase()` 做大小写不敏感比对。未登录跳 `/auth/login`，角色不匹配跳 `/`。校验通过后 `setUser(currentUser)` 并 `setLoading(false)`，页面根据 `loading` 渲染骨架屏，根据 `user` 决定是否展示敏感内容。`useMerchantAuth()` 和 `useAdminAuth()` 是对 `useAuth('merchant')` 和 `useAuth('admin')` 的语义化封装，减少调用方的字符串拼写错误。

6.1.3 权限精细化校验 `app/api/hotels/[id]/route.tsx`（PUT 方法）

做了什么：在 JWT 角色校验之上，对酒店状态变更操作额外验证细粒度权限，区分“商户下线自己的酒店”与“管理员审核/拒绝酒店”两种场景。

怎么做的：通过 Prisma 关联查询当前用户的完整权限列表（`role → rolePermission → permission`），检查是否含 `HOTEL_AUDIT` 或 `HOTEL_DELETE` 权限名称。下线操作允许本人或有 `HOTEL_AUDIT` 权限者执行；审核通过/拒绝仅限 `HOTEL_AUDIT` 权限。状态变更与 `HotelAuditLog` 写入在同一 `prisma.$transaction` 内原子完成，保证日志不漏不重。

6.1.4 权限检查性能优化

核心文件：`app/api/utils/permissions.ts#L7-L71`（`PermissionCache` 类）、`app/api/utils/permissions.ts#L110-L142`（批量权限检查 API）

业务价值：解决商户管理端频繁权限查询的性能瓶颈。管理员在酒店审核、用户管理等页面操作时，每个 API 请求都需要校验细粒度权限（HOTEL_AUDIT、USER_MANAGE 等），未优化前每次查询需 35-50ms，高频操作下响应延迟明显。

做了什么：实现 15 分钟 TTL 内存缓存机制，将用户权限集合缓存在应用内存中，避免每次权限检查都进行深层数据库查询（`role → rolePermission → permission` 三表关联）。

怎么做的：

`PermissionCache` 单例类（`app/api/utils/permissions.ts#L7-L71`）管理缓存层，使用 `Map` + `Set` 数据结构存储。`checkPermission(userId, permissionName)` 函数实现三步查询逻辑：① 尝试从缓存获取权限集合；② 缓存未命中时通过 Prisma 查询数据库并写入缓存；③ 使用 `Set.has()` 高效检查权限（O(1) 时间复杂度）。

数据库查询优化：仅 select 必需字段（`permission.name`），减少数据传输量。缓存过期策略采用被动检查模式——每次 `get()` 时比对时间戳，超过 15 分钟自动失效并返回 `null`。提供 `clearPermissionCache(userId)` 接口，在用户角色变更时主动清除缓存，保证数据一致性。

批量权限检查 API：`checkAnyPermission()`（OR 逻辑）和 `checkAllPermissions()`（AND 逻辑），均复用缓存层。

性能收益（本地测试环境）：权限检查响应时间从 35-50ms 降至 $<$ 2ms（缓存命中场景），缓存命中率约 90%；管理后台操作流畅度明显提升，特别是酒店列表批量操作场景。

6.2 预订事务实现

6.2.1 相关文件

- 小程序端入口：`trip_front_end_taro/src/components/BookingConfirm/index.jsx`（用户确认界面）
- 小程序服务层：`trip_front_end_taro/src/services/booking.js`（发起 HTTP 请求）
- 后端接口：`app/api/bookings/route.tsx#L208-L337`（POST 方法，核心事务逻辑：逐日检查库存并原子递增 booked）
- 数据库模型：`prisma/schema.prisma`（`RoomAvailability` 唯一约束定义）

6.2.2 做了什么

用户在 `BookingConfirm` 组件填写入住人信息并确认后，系统在数据库事务内完成库存校验、库存占用与订单创建三件事，保证高并发下不超卖。

6.2.3 怎么做的

后端 POST 接口先调用 `verifyAuth()` 确认用户已登录，再校验日期合法性，然后开启 `prisma.$transaction`。事务内采用**批量查询 + 逐日原子更新**的优化策略：

1. **批量查询阶段**（3 次数据库操作）：① 一次性 `findMany` 查询入住区间内所有日期的 `RoomAvailability` 记录；② 对缺失日期执行 `createMany({ skipDuplicates: true })` 批量初始化；③ 再次 `findMany` 获取完整可用性数据。在内存中遍历验证每日状态（`isClosed`、`booked >= quota`），累加每日价格计算总价——该阶段将原先 N×2 次查询优化为固定 3 次。
2. **逐日原子更新阶段**（N 次数据库操作）：对每一天使用条件 `updateMany({ where: { booked: { lt: quota } }, data: { booked: { increment: 1 } } })` 原子递增已订数，保证并发安全。如 `updated.count === 0` 则说明被其他事务抢占，抛错回滚。

综合来看，7 晚预订从原先 21 次数据库操作（每天 upsert + findUnique + updateMany）降至 10 次（3 次批量 + 7 次原子更新），减少约 52% 的数据库交互。MySQL InnoDB 行锁在事务内保护同一日期的并发写操作互斥，任意步骤失败则所有 `booked` 变更一并回滚，从数据库层彻底消除超卖。

6.3 AI 推荐实现

6.3.1 相关文件

- 小程序悬浮入口：`trip_front_end_taro/src/components/AiChatWidget/index.jsx`（流式渲染 UI）
- Web 端悬浮入口：`app/components/AiChatWidget.tsx`（同功能）
- 后端接口：`app/api/ai/recommend/route.ts`（两阶段 LLM 调用）
- 模型配置：`app/lib/openai.ts`（ChatOpenAI 实例，从 `.env` 读取 API Key 和 Base URL）

6.3.2 做了什么

用户以自然语言描述需求（如“找北京带早餐性价比高的酒店”），系统自动理解意图、查询匹配酒店，并以流式打字机效果逐字返回推荐内容，支持多轮对话。

6.3.3 怎么做的

后端接口分两个阶段处理：

第一阶段——结构化意图提取：使用 LangChain 的 `model.withStructuredOutput(ZodSchema)` 以温度 0（确定性输出）调用模型，从最近 3 条对话消息中提取结构化搜索条件（目的地、价格区间、关键词、设施、排序方式）。Zod Schema（`app/api/ai/recommend/route.ts#L9-L16`）同时约束输出结构并与 TypeScript 类型同步。

第二阶段——流式推荐生成：用提取出的条件通过 Prisma 查询数据库，最多取 20 条已发布酒店，按价格/评分排序后取前 5 条，将酒店信息注入 System Prompt。再以温度 0.7（增加回复多样性）调用模型，将 LangChain 返回的 AsyncIterable 逐 chunk 写入 Web 标准 `ReadableStream`，以 `text/plain` 流式响应返回。前端 `AiChatWidget` 通过 `response.body.getReader()` 循环读取每个文字片段，实时拼接到当前消息气泡，实现逐字打印效果，无需额外 SSE 库。

前端 AiChatWidget 两端实现对比：

Web 端（`app/components/AiChatWidget.tsx`）基于 Ant Design `<FloatButton>` 固定定位于页面右下角，聊天窗口以 `<Card>` 浮层呈现。消息类型为 `{ role: 'user' | 'assistant', content: string }`，通过 `fetch` 接收流式响应——先向 `messages` 数组追加一条 `content: ''` 的占位 assistant 消息，再在 `while(true)` 循环内用 `TextDecoder` 解码每个 `Uint8Array` chunk 并追加到最后一条消息，每次更新触发 React 重渲染实现逐字效果。`Enter` 键直接发送（`Shift+Enter` 换行）。自动滚动通过 `useRef` 绑定末尾 `<div>` 并调用 `scrollIntoView({ behavior: 'smooth' })`。

小程序端（`trip_front_end_taro/src/components/AiChatWidget/index.jsx`）受微信小程序环境限制，`Taro.request` 不支持流式读取，因此退化为单次请求模式——发送后等待完整响应再渲染。悬浮按钮通过 `position: fixed` + `onTouchStart/Move/End` 三个触摸事件实现

可拖拽效果：以 3px 位移阈值区分“点击”与“拖动”，拖动时计算新坐标并用 `Math.min/max` 限制在屏幕边界内，未发生移动的 `TouchEnd` 触发打开聊天窗口。消息列表用 `<ScrollView scrollIntoView={lastMsgId}>` 自动滚动至最新消息。

7 Web 端实现

7.1 管理登录注册实现

7.1.1 相关文件

- 登录页：`app/auth/login/page.tsx`
- 注册页：`app/auth/register/page.tsx`
- 认证服务：`app/services/auth.ts`（`login`、`register`、`saveAuth`）

7.1.2 功能概述

管理端支持两种角色——**商户（Merchant）**和**管理员（Admin）**。注册时用户通过下拉菜单主动选择角色；登录后系统读取 JWT 中的角色信息，自动跳转至对应后台入口，无需用户手动选择页面。

7.1.3 注册页角色选择

注册表单使用 Ant Design `<Select>` 组件让用户在注册时选择身份：

来源：`app/auth/register/page.tsx`

```
<Form.Item
  name="role"
  rules={[{ required: true, message: '请选择角色' }]}
>
  <Select placeholder="选择角色">
    <Option value="merchant">商户</Option>
    <Option value="user">普通用户</Option>
  </Select>
</Form.Item>
```

选择的 `role` 字段随注册请求发往 `POST /api/auth/register`，后端在 `Role` 表查找对应角色并关联用户。管理员账号在数据库种子中预置，不在注册页暴露选项。

7.1.4 登录后角色自动跳转

登录成功后，系统从 JWT 响应中读取 `user.role.name`，依据角色自动跳转至对应后台，无需用户手动导航：

来源：`app/auth/login/page.tsx`

```
const roleName = result.user.role?.name?.toLowerCase();

if (roleName === 'merchant') {
  router.push('/merchant/hotels');          // 商户 → 酒店管理
} else if (roleName === 'admin' || roleName === 'administrator') {
  router.push('/admin/review');             // 管理员 → 审核中心
} else {
  router.push('/');                         // 默认 → 首页
}
```

此机制满足题目要求的“登录时自动识别角色”功能点，商户和管理员进入系统后看到完全不同的操作界面。

7.2 商户酒店录入与编辑实现

7.2.1 相关文件

- 商户酒店管理页：`app/merchant/hotels/page.tsx`
- 地图选点组件：`app/components/TencentMapSelector.tsx`
- 后端接口：`app/api/hotels/route.tsx`（POST 创建）、`app/api/hotels/[id]/route.tsx`（PUT 更新）

7.2.2 功能概述

商户在管理后台可新建酒店或编辑已有酒店信息，所有字段在同一弹窗 Modal 内录入，保存后数据实时同步至用户端搜索结果（仅需管理员审核通过）。

7.2.3 表单字段说明

字段	必填	说明
`nameZh`（酒店中文名）	✓	用户端主要展示名称，列表页/详情页均显示此字段
`nameEn`（英文名）	—	对应题目“中英文展示”要求，英文版 UI 展示
`starRating`（星级）	✓	1–5 星下拉选择，影响 FilterPanel 星级筛选结果
`locationId`（城市）	✓	下拉选择系统预置城市，关联 `Location` 表
`address`（详细地址）	✓	地址字符串，用户端展示
地理位置（经纬度）	✓	通过 `TencentMapSelector` 点击地图自动填充 `latitude/longitude`
`openingYear`（开业年份）	✓	`DatePicker` 选择年份，满足“开业日期”必填要求
`hotelTags`（标签）	—	多选标签（如“亲子友好”/“免费停车”），对应题目周边信息可选项
`description`（描述）	—	酒店介绍文字
图片（最多 8 张）	—	Upload 组件上传至 `/api/upload`，返回 URL 存储
`rooms`（房型列表）	—	可动态增删房型，每个房型含名称/价格/库存/折扣/设施/图片

7.2.4 腾讯地图坐标拾取

`TencentMapSelector` 组件（`app/components/TencentMapSelector.tsx`）内嵌腾讯地图 JS API，商户点击地图上任意位置后，组件通过 `onSelect` 回调将经纬度写入表单隐藏字段：

```
<TencentMapSelector
  latitude={getFieldValue('latitude')}
  longitude={getFieldValue('longitude')}
  onSelect={(loc) => setFieldsValue({
    latitude: loc.latitude,
    longitude: loc.longitude
  })}
/>
```

坐标提交后存储于 `Hotel.latitude/Hotel.longitude`，用户端酒店详情页“在地图上查看”功能据此定位。

7.3 AI 聊天助手实现

7.3.1 相关文件

- Web 组件（185 行）：`app/components/AiChatWidget.tsx`
- 小程序组件：`trip_front_end_taro/src/components/AiChatWidget/index.jsx`
- 后端接口：`app/api/ai/recommend/route.ts`（两阶段 LLM 调用）

- 模型配置：`app/lib/openai.ts`（ChatOpenAI 实例）

7.3.2 功能概述

悬浮聊天窗口固定于页面右下角（Web 端）或可拖拽悬浮（小程序端），用户以自然语言描述住宿需求，系统以流式打字机效果逐字返回 AI 推荐结果，支持多轮对话。

7.3.3 流式响应接收（SSE over Fetch）

Web 端通过 Fetch API 接收后端以 `text/plain` 格式返回的 `ReadableStream`，实现无 SSE 库的流式渲染：

来源：`app/components/AiChatWidget.tsx`

```
const response = await fetch('/api/ai/recommend', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ messages: [...messages, userMsg] }),
})

const reader = response.body!.getReader()
const decoder = new TextDecoder()

// 创建空消息占位，后续 chunk 追加到此
let assistantMsg: Message = { role: 'assistant', content: '' }
setMessages(prev => [...prev, assistantMsg])

while (true) {
  const { done, value } = await reader.read()
  if (done) break
  // 解码二进制 chunk → 追加文本
  assistantMsg.content += decoder.decode(value, { stream: true })
  // 触发 React 重渲染，实现逐字打印效果
  setMessages(prev => {
    const next = [...prev]
    next[next.length - 1] = { ...assistantMsg }
    return next
  })
}
```

后端使用 LangChain 的 `AsyncIterable` 逐 chunk 写入 `ReadableStream`，前端无需轮询或 EventSource，纯 `response.body.getReader()` 即可实现实时流式展示。

7.3.4 消息状态管理

消息列表类型为 `Message[] = { role: 'user' | 'assistant', content: string }[]`。流式接收过程中：

- 发送前：`loading = true`（禁用发送按钮，防止重复提交）
- 追加占位：向数组末尾追加 `{ role: 'assistant', content: '' }`
- 逐 chunk 更新：通过不可变更新（展开运算符）替换最后一条消息，每次 setState 触发重渲染

- 完成后：`loading = false`，发送按钮恢复

自动滚动通过 `useRef` 绑定消息列表末尾 `<div>` 并调用 `scrollIntoView({ behavior: 'smooth' })` 实现，每次 messages 变化后触发。

7.3.5 小程序端降级方案

微信小程序的 `Taro.request` 不支持 `ReadableStream` 流式读取，因此小程序端退化为单次请求模式——发送消息后等待完整响应再一次性渲染。悬浮按钮通过 `position: fixed` + `onTouchStart/Move/End` 三个触摸事件实现可拖拽：以 3px 位移阈值区分"点击"与"拖动"，拖动时计算新坐标并以 `Math.min/max` 约束在屏幕边界内，未产生位移的 `TouchEnd` 触发打开聊天窗口。

7.4 前端 HTTP 客户端实现

7.4.1 相关文件

- Web 端封装：`app/lib/request.ts`（Axios 实例 + 拦截器）
- 小程序端封装：`trip_front_end_taro/src/services/request.js`（Taro.request 封装）
- Web 业务服务层：`app/services/` 下的 `app/services/auth.ts`、`app/services/hotel.ts`、`app/services/booking.ts`、`app/services/room.ts`、`app/services/review.ts`、`app/services/admin.ts`
- 小程序业务服务层：`trip_front_end_taro/src/services/` 下各 `.js` 文件

7.4.2 做了什么

两端各自封装一个 HTTP 客户端单例，所有业务 Service 函数只需调用 `get/post/put/del` 四个类型化方法，Token 注入、响应解包、401 无感刷新均由客户端统一处理，业务代码零冗余。

7.4.3 怎么做的

Web 端（`app/lib/request.ts`）：以 `axios.create({ baseURL: '/api', timeout: 10000 })` 创建实例，挂载两个拦截器。请求拦截器通过 `authStorage.getToken()` 统一读取 Token（该工具自动处理 SSR 环境守卫），写入 `Authorization: Bearer <token>` Header。响应拦截器的成功回调直接 `return response.data`，让调用方无需每次解包；失败回调遇到 401 时，先尝试用 `refreshToken` 调用 `/api/auth/refresh` 无感换取新 `accessToken`——通过 `isRefreshing` 布尔锁 + `failedQueue` 队列保证并发安全（多个请求同时 401 时只发一次 refresh，其余请求排队等待新 Token 后自动重发）；refresh 成功后调用 `authStorage.saveAuth(newToken, newRefreshToken, user)` 统一写入存储，原始请求自动重试；refresh 失败则调用 `authStorage.clearAuth()` 统一清除认证数据并 `window.location.href = '/auth/login'` 跳转登录页。最终导出泛型化的 `get<T>/post<T>/put<T>/del<T>` 方法，调用方可直接标注返回类型。

小程序端（`trip_front_end_taro/src/services/request.js`）：镜像相同结构，用 `Taro.getStorageSync('token')` 同步读取 Token，通过 `Taro.request` 发起请求，统一超时 10 秒。响应处理分三条路径：`statusCode === 200` 直接返回 `res.data`；`statusCode === 401` 清除存储并 `Taro.navigateTo({ url: '/pages/login/index' })`；其他错误码从 `res.data.error` 提取错误信息并以 `Taro.showToast` 展示给用户。网络错误（`errMsg` 包含 `request:fail`）和

超时（包含 timeout）分别弹出对应提示。GET 请求额外实现了 Query 参数序列化：过滤 null/undefined 值后 encodeURIComponent 编码拼接为查询字符串。

7.4.4 认证状态统一管理 app/lib/auth-storage.ts

做了什么：封装所有 localStorage 认证相关操作，消除代码重复，提供类型安全的 API，实现认证状态的集中式管理。

怎么做的：

导出 authStorage 对象，包含完整的认证状态管理方法：getToken() / setToken(token) 读写访问令牌，getRefreshToken() / setRefreshToken(token) 管理刷新令牌，getUser() / setUser(user) 处理用户信息的 JSON 序列化与反序列化，saveAuth(token, refreshToken, user) 批量写入认证三元组，clearAuth() 一键清除所有认证数据。

所有方法内部统一处理 SSR 环境守卫（typeof window !== 'undefined'），读取操作在服务端渲染时自动返回 null，避免 localStorage is not defined 错误。用户对象的 getUser() 自动捕获 JSON 解析异常并返回 null，保证存储损坏时不抛异常。

辅助校验方法：isAuthenticated() 检查 Token 是否存在（返回布尔值），hasRole(role) 检查当前用户是否拥有指定角色（支持大小写不敏感比对）。STORAGE_KEYS 常量从 app/constants/index.ts 导入，保证键名全局统一。

使用场景：

- app/lib/request.ts — 请求拦截器通过 authStorage.getToken() 读取 Token，响应拦截器通过 authStorage.saveAuth() 写入刷新后的 Token；
- app/services/auth.ts — 登录成功后调用 authStorage.saveAuth(accessToken, refreshToken, user) 一次性写入所有认证数据，登出时调用 authStorage.clearAuth() 清除；
- app/hooks/useAuth.ts — 前端路由保护 Hook 通过 authStorage.isAuthenticated() 检查登录态，通过 authStorage.getUser() 获取当前用户；
- 总计约 8 个文件共享该工具，修改一处即全局生效，彻底消除 localStorage.getItem('token') 等原始调用散布在代码各处的问题。

7.4.5 完整调用示例：Taro 小程序加载酒店列表

以下展示从前端发起请求到后端返回数据的完整数据流转，演示三端共用 API 架构的实际运作机制。

场景：用户在小程序酒店列表页搜索“上海”地区的酒店，系统需从后端获取符合条件的数据并渲染。

① 前端页面层（trip_front_end_taro/src/pages/hotelList/index.jsx#L238-L243）

页面组件调用 Service 层的业务函数，传入筛选参数：

```
const loadHotels = async (params = {}) => {
  const res = await getHotels({
    locationId: params.locationId,  // 城市 ID
```

```
    keyword: params.keyword,        // 搜索关键词
    type: params.type,              // 酒店类型
    page: 1,
  });
  setHotels(res.data);  // res.data 已是解析后的酒店数组
};
```

② **Service 业务层**（`trip_front_end_taro/src/services/hotel.js#L23-L55`）

Service 层负责参数清洗和 API 路径映射，过滤 `null/undefined` 值后调用请求封装：

```
export const getHotels = async (params = {}) => {
  const queryParams = {};
  if (params.locationId) queryParams.locationId = params.locationId;
  if (params.keyword) queryParams.keyword = params.keyword;
  if (params.type) queryParams.type = params.type;
  // 调用请求封装层的 GET 方法
  return get('/hotels', queryParams);
};
```

③ **请求封装层**（`trip_front_end_taro/src/services/request.js#L22-L53`）

请求封装层自动从本地存储读取 Token，注入 `Authorization` Header，发起 HTTP 请求：

```
function request(url, options = {}) {
  const token = Taro.getStorageSync('token');
  const header = {
    'Content-Type': 'application/json',
    ...options.header,
  };
  if (token) header.Authorization = `Bearer ${token}`;

  return Taro.request({
    url: BASE_URL + url,  // http://localhost:3000/api/hotels?locationId=1&
keyword=上海
    method: options.method || 'GET',
    header,
    timeout: 10000,
  }).then(res => {
    if (res.statusCode === 401) {
      Taro.removeStorageSync('token');
      Taro.navigateTo({ url: '/pages/login/index' });
      return Promise.reject(new Error('未授权'));
    }
    return res.data;  // 自动解包，返回 { success: true, data: [...] }
  });
}
```

④ **Next.js API Routes**（`app/api/hotels/route.tsx#L167-L240`）

后端 API 解析查询参数，构建 Prisma 查询条件，查询数据库并返回 JSON：

```
export async function GET(request: NextRequest) {
  const { searchParams } = new URL(request.url);
  const locationId = searchParams.get('locationId');
  const keyword = searchParams.get('keyword');
  const type = searchParams.get('type');

  const where: HotelWhereInput = {};
  if (locationId) where.locationId = parseInt(locationId);
  if (type) where.type = type;
  if (keyword) {
    where.OR = [
      { nameZh: { contains: keyword } },
      { address: { contains: keyword } },
    ];
  }

  const hotels = await prisma.hotel.findMany({
    where,
    include: {
      location: true,
      merchant: true,
      roomTypes: true,
      hotelTags: { include: { tag: true } },
    },
    skip: (page - 1) * limit,
    take: limit,
  });

  return NextResponse.json({
    success: true,
    data: hotels,
    pagination: { page, limit, total: await prisma.hotel.count({ where }) },
  });
}
```

⑤ 响应数据格式

后端返回的 JSON 响应自动被请求封装层解包，业务代码直接获得 `data` 字段：

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "nameZh": "上海浦东希尔顿酒店",
      "address": "浦东新区世纪大道XXX号",
```

```
      "starRating": 5,
      "location": { "id": 1, "name": "上海" },
      "roomTypes": [
        { "id": 1, "name": "豪华大床房", "price": 899, "stock": 10 }
      ],
      "hotelTags": [
        { "tag": { "id": 1, "name": "商务出行" } }
      ]
    }
  ],
  "pagination": { "page": 1, "limit": 10, "total": 25 }
}
```

关键优势：整个流程中，Token 自动注入、响应解包、401 跳转登录、网络错误提示均由请求封装层统一处理，页面组件和 Service 层的业务代码零冗余。Web 端的 `app/lib/request.ts` 采用相同架构，但使用 Axios 拦截器实现更复杂的 Token 自动刷新机制（`app/lib/request.ts#L86-L133`），多个并发 401 请求通过 `failedQueue` 队列合并为一次刷新操作，避免重复调用 `/auth/refresh` 接口。

7.5 商户数据看板实现

7.5.1 相关文件

- 页面组件：`app/merchant/dashboard/page.tsx`（数据聚合 + 图表渲染）
- 数据来源：`app/services/booking.ts`（`getMyBookings` 拉取商户全量预订）
- 图表库：Recharts 3.7.0（`LineChart`、`BarChart`、`PieChart`）

7.5.2 做了什么

商户进入看板页面后，自动展示今日预订数、本月营收、入住率三个 KPI 指标，以及近 7 天营收折线图、各月预订柱状图和房型分布饼图。

7.5.3 怎么做的

页面加载时调用 `getMyBookings` 一次性拉取当前商户的全量预订数据，随后在客户端纯 JavaScript 计算各项指标：今日预订通过入住日期与当天比对筛选；月营收对状态为 `completed` 或 `checked_out` 且本月创建的预订累加 `totalPrice`；7 天趋势数据按日期分桶聚合；房型分布将预订按 `roomType.name` 分组计数。所有聚合结果直接作为 Recharts 各图表的 `data` prop 传入，组件声明式渲染为 SVG 图表。此方案无需额外统计接口，但预订量极大时可考虑改为服务端聚合。

7.6 深色模式实现

7.6.1 相关文件

- 小程序主题 Hook：`trip_front_end_taro/src/utils/useTheme.js`
- 小程序主题工具函数：`trip_front_end_taro/src/utils/theme.js#L21-L58`（`TOKENS` 深色/浅色主题色定义）、`resolveTheme` / `getThemeCssVars` / `applyNativeTheme`
- 小程序使用方：各页面组件（`home`、`hotelDetail`、`mine` 等）根 View 绑定 `style={cssVars}`

- Web 端：`app/providers.tsx`（`next-themes` ThemeProvider）、`app/components/Navbar.tsx`（切换按钮）

7.6.2 做了什么

小程序端和 Web 端均支持深色/浅色主题切换，且多页面间状态实时同步。小程序通过 CSS 变量注入方式实现全局主题切换，无需给每个子组件单独传 props。

7.6.3 怎么做的

小程序端：`useTheme()` Hook 以懒初始化方式（`useState(() => ...)` 避免重复计算）读取 `resolveTheme()` 的结果——该函数优先读 `Taro.getStorageSync('theme')` 中的用户偏好，缺失时回退到 `Taro.getSystemInfoSync().theme` 系统设置。Hook 返回三个值：`cssVars`（一段 CSS 变量字符串，如 `--color-bg:#1a1a1a;--color-text:#fff;...`）、`isDark` 布尔值、以及 `tokens`（已解析的主题色键值对对象，如 `{ '--color-primary': '#1677ff', '--color-text-primary': '#1a1a1a', ... }`）。

`cssVars` 用于 CSS 变量注入，各页面将其绑定到根 `<View style={cssVars}>`，子组件通过 `var(--color-xxx)` 继承。`tokens` 用于 JS 层面需要动态颜色的场景——典型用例是 SVG Icon 组件的 `color` 属性：由于 Taro 小程序中 SVG 的 `fill` 属性不支持 CSS 变量，Icon 组件通过 `tokens['--color-text-primary']` 获取已解析的十六进制色值，注入 SVG data URI。这一设计保证了 `theme.js` 中的 `TOKENS` 对象是颜色的唯一真相源（Single Source of Truth），避免各组件硬编码 `isDark ? '#hex1' : '#hex2'` 三元表达式。

主题同步通过两条路径保证：① Taro 的 `useDidShow` 生命周期钩子在每次页面重新进入前台时重新计算并应用主题（处理从“我的”页切换主题后返回其他页的场景）；② `useEffect` 内监听 `Taro.eventCenter.on('themeChanged', handler)`，当用户在“我的”页手动切换时，`app.js` 广播该事件，所有已挂载页面的 Handler 同步更新状态，退出时 `off` 清理防止内存泄漏。

Web 端：由 `next-themes` 的 `ThemeProvider` 统一管理，`data-theme` 属性写入 `<html>` 标签，`globals.css` 通过 `[data-theme='dark']` 选择器覆写 CSS 变量。Navbar 调用 `useTheme().setTheme()` 完成切换，无需额外状态管理。

7.7 Icon 图标组件与骨架屏

7.7.1 相关文件

- 图标组件：`trip_front_end_taro/src/components/Icon/index.jsx`
- 图标 path data：`trip_front_end_taro/src/utils/icons.js`（借鉴 Phosphor Icons 开源库）
- 骨架屏组件：`trip_front_end_taro/src/components/Skeleton/index.jsx`
- 骨架屏样式：`trip_front_end_taro/src/components/Skeleton/index.css`

7.7.2 做了什么

Icon 组件统一了小程序端全部页面的图标渲染（40+ 处调用），替代了之前散落的 emoji 和 unicode 字符。骨架屏组件为酒店卡片、酒店详情、订单卡片三种场景提供内容占位加载态，提升感知加载速度。

7.7.3 怎么做的

Icon：图标 SVG path data 借鉴自 Phosphor Icons 开源图标库（MIT 协议），存放于 `icons.js` 的 `ICON_PATHS` 对象（20+ 个图标，涵盖导航、搜索、收藏、日历等分类）。`createIconUri(pathData, color)` 将 path data 拼接为完整 SVG 字符串后 `encodeURIComponent` 生成 data URI；`getIconUri(name, color)` 封装了名称查找。Icon 组件内部用 `useMemo` 缓存生成结果，通过 Taro `<Image>` 的 `src` 属性渲染，绕开了小程序不支持内联 SVG 的限制。

图标颜色由调用方传入，统一使用 `tokens['--color-text-primary']` 等已解析色值（而非 CSS 变量字符串），保证深色/浅色模式下图标颜色跟随主题自动切换。

Skeleton：组件接收 `type` 参数，按场景渲染不同骨架布局——`hotelCard` 模拟卡片列表（图片+标题+标签+价格），`hotelDetail` 模拟详情页（Banner+设施行+房型列表），`orderCard` 模拟订单卡片（头部+信息行+按钮）。所有占位块通过 `.skeleton-shimmer` CSS 类添加从左到右的渐变光扫动画（`@keyframes shimmer`），`count` 参数控制列表类骨架的重复数量。

7.8 管理员酒店审核实现

7.8.1 相关文件

- 审核页面：`app/admin/review/page.tsx`
- 后端接口：`app/api/hotels/[id]/route.tsx`（PUT，状态变更 + AuditLog 原子写入）
- 权限控制：`app/api/utils/permissions.ts`（`HOTEL_AUDIT` 权限检查）

7.8.2 做了什么

管理员进入审核页后看到所有酒店的状态表格（待审核 / 已发布 / 已拒绝 / 已下线），可对每家酒店执行：查看详情（侧边 Drawer 展示图片、基本信息、房型列表）、通过审核、拒绝（填写拒绝理由）、下线、恢复上线。所有操作均有二次确认弹窗防止误操作。

7.8.3 怎么做的

表格以 Ant Design `<Table>` 渲染，通过 `columns` 的 `render` 函数根据酒店当前 `status` 字段动态展示不同操作按钮（`pending` 显示通过/拒绝；`published` 显示下线；`offline` 显示恢复）。点击“查看”将目标酒店写入 `selectedHotel` 并打开 `<Drawer>`，Drawer 内用 `<Descriptions>` 瀑布布局展示详情，`<Image.PreviewGroup>` 支持图片放大预览。拒绝操作弹出 `<Modal>` 要求填写拒绝理由，提交后调用 `rejectHotel(id, reason)` 将 `status` 设为 `rejected` 并附带理由；服务端在同一 `prisma.$transaction` 内同步写入 `HotelAuditLog`，保证审核记录不丢失。审核通过调用 `approveHotel(id)` 将状态改为 `published`，酒店立即出现在用户端搜索结果中。

拒绝原因展示：管理员填写的拒绝理由存储于 `Hotel.rejectionReason` 字段，商户在酒店管理列表中可看到被拒绝酒店对应的原因文本，据此修改信息后重新提交审核。

下线与恢复（状态机替代软删除）：系统以状态机模型替代传统 `isDeleted` 软删除标志——酒店下线（`offline`）后全部数据完整保留，管理员可通过“恢复上线”按钮将其重新置为 `published` 状态，满足题目“下线为软删除、可恢复”的功能要求。状态流转图：`pending → published ↔ offline`（双向可逆），`pending → rejected`（可重新提交）。

7.9 管理员优惠券管理实现

7.9.1 相关文件

- 优惠券管理页：`app/admin/coupons/page.tsx`
- 数据服务层：`app/services/coupon.ts`（`getCoupons`、`createCoupon`、`updateCoupon`、`deleteCoupon`）
- 后端接口：`app/api/coupons/route.ts`（GET/POST）、`app/api/coupons/[id]/route.ts`（PUT/DELETE）

7.9.2 做了什么

管理员进入优惠券管理页后，以表格形式查看系统全部优惠券（代码、名称、折扣金额、最低消费、兑换所需积分、有效期），可执行新增、编辑和删除操作。

7.9.3 怎么做的

页面使用 `useAdminAuth` Hook 校验当前用户角色，非 ADMIN 自动重定向。表格以 Ant Design `<Table>` 渲染优惠券列表，列包括：优惠券代码（`<Tag color="blue">`）、名称、折扣金额与最低消费（组合显示"减 X 元（满 Y 可用）"）、兑换积分、有效期范围、动态状态标签（根据当前日期判定"生效中"/"未开始"/"已过期"），以及编辑/删除操作按钮。

新增和编辑共用同一个 `<Modal>` + `<Form>` 弹窗（通过 `editingCoupon` 状态区分模式），使用 `Form.useForm()` 管理表单状态。表单字段包括：优惠券代码（`Input`，必填）、名称（`Input`，必填）、描述（`TextArea`，可选）、折扣金额（`InputNumber`，必填）、最低消费（`InputNumber`，可选）、兑换所需积分（`InputNumber`，可选，默认 0 表示免费领取）、有效期（`DatePicker.RangePicker`，必填）。提交时通过 `form.validateFields()` 校验后调用 `createCoupon` 或 `updateCoupon`，成功后重新拉取列表。删除操作通过 `<Popconfirm>` 二次确认后调用 `deleteCoupon`，成功后乐观更新本地 state。反馈统一使用 `App.useApp()` 的 `message` 组件。

7.10 商户职员管理实现

7.10.1 相关文件

- 职员管理页：`app/merchant/staff/page.tsx`
- 数据服务层：`app/services/staff.ts`（`getMyStaff`、`createStaff`、`updateStaff`、`deleteStaff`）
- 后端接口：`app/api/users/route.tsx`（GET `?myStaff=true`）、`app/api/users/[id]/route.tsx`（PUT/DELETE，商户管理职员权限）
- 注册接口：`app/api/auth/register/route.tsx`（POST，`role=staff` + `merchantId` 创建职员）

7.10.2 做了什么

商户可在侧边栏"职员管理"页面中查看、添加、编辑和删除所属职员账号。职员角色（STAFF）登录后共享同一套商户管理系统（仪表盘、酒店、日历、预订），但侧边栏不显示"职员管理"和"优惠券管理"两项菜单，实现基于角色的功能隔离。

7.10.3 怎么做的

后端权限扩展：GET `/api/users` 新增 `?myStaff=true` 查询路径，校验调用者角色为 MERCHANT 后返回 `merchantId` 等于当前用户 ID 的所有职员记录。PUT/DELETE `/api/`

users/[id] 在原有“本人或管理员”权限基础上，新增商户管理自身职员的判断逻辑——通过 targetUser.merchantId === currentUserId 确认归属关系后放行操作，无需在权限表中额外配置。

前端实现：页面使用 getStoredUser() 获取当前用户角色，非 MERCHANT 展示 <Empty description="无权访问"> 占位。表格以 Ant Design <Table> 渲染职员列表（姓名、邮箱、手机、创建时间、操作列）。添加职员时通过 <Modal> + <Form> 收集邮箱、密码、姓名、手机字段，调用 createStaff() 封装的注册接口（自动附加 role: 'staff' 和 merchantId）。编辑仅允许修改姓名和手机，删除通过 <Popconfirm> 二次确认。

菜单隔离：layout.tsx 在构建 menuItems 时通过 isMerchant 判断动态展开“职员管理”和“优惠券管理”菜单项（...(isMerchant ? [...] : [])），STAFF 角色登录后仅看到仪表盘、酒店管理、房间日历、预订管理四个入口。

7.11 商户个人信息实现

7.11.1 相关文件

- 个人信息页：app/merchant/profile/page.tsx
- 数据服务层：app/services/staff.ts（getProfile、updateProfile、changePassword）
- 后端接口：app/api/users/profile/route.tsx（GET 当前用户资料）、app/api/users/[id]/route.tsx（PUT 更新资料/密码）

7.11.2 做了什么

商户和职员均可通过右上角头像下拉菜单的“个人信息”选项进入个人资料页，查看邮箱、姓名、手机、角色、积分、注册时间等信息，并支持编辑基本信息和修改密码。

7.11.3 怎么做的

页面加载时调用 getProfile() 从服务端获取最新用户数据，以 <Card> + <Descriptions> 展示。编辑信息和修改密码分别使用独立的 <Modal> + <Form>，各自绑定 Form.useForm() 实例。

编辑信息：Modal 打开时通过 form.setFieldsValue() 回填当前姓名和手机，提交后调用 updateProfile(userId, { name, phone }) 更新，成功后同步更新页面 state 和 authStorage.setUser()（保证 Header 中的用户名实时刷新）。

修改密码：后端 PUT /api/users/[id] 新增 oldPassword + newPassword 字段支持，仅允许本人操作（isSelf 校验）。服务端通过 bcrypt.compare() 验证旧密码正确性，不匹配返回 400；验证通过后 bcrypt.hash(newPassword, 10) 加密存储。前端表单包含旧密码、新密码、确认密码三个字段，确认密码通过 dependencies + validator 实时校验一致性。

7.12 商户优惠券管理实现

7.12.1 相关文件

- 优惠券管理页：app/merchant/coupons/page.tsx
- 数据服务层：app/services/coupon.ts（getCoupons、createCoupon、updateCoupon、deleteCoupon）

- 后端接口：`app/api/coupons/route.ts`（GET/POST）、`app/api/coupons/[id]/route.ts`（PUT/DELETE）

7.12.2 做了什么

商户可在侧边栏“优惠券管理”页面中管理系统优惠券，与管理员端共享同一套后端接口和数据服务层，仅前端入口和角色校验不同。

7.12.3 怎么做的

页面结构与管理员优惠券页保持一致（Ant Design `<Table>` + `<Modal>` + `<Form>` 模式），角色校验通过 `getStoredUser()` 判断是否为 MERCHANT，非商户展示 `<Empty>` 占位。表格列新增“状态”列，根据当前日期与 `validFrom`/`validTo` 对比动态显示 `<Tag color="green">生效中</Tag>`、`<Tag color="orange">未开始</Tag>` 或 `<Tag color="default">已过期</Tag>`。有效期输入使用 `<DatePicker.RangePicker>` 替代原生日期选择器，提升交互体验。

8 小程序端实现

8.1 首页搜索与筛选实现

8.1.1 相关文件

- 首页（779 行）：`trip_front_end_taro/src/pages/home/index.jsx`
- 搜索建议组件：`trip_front_end_taro/src/components/SearchSuggestion/index.jsx`
- 日历组件：`trip_front_end_taro/src/components/Calendar/index.jsx`
- 数据接口：`trip_front_end_taro/src/services/hotel.js`（`getLocations`、`getTags`）

8.1.2 功能概述

用户进入首页后可选择住宿类型（国内 / 海外 / 钟点房 / 民宿）、目标城市、入离日期和筛选条件（价格区间、设施标签），点击搜索跳转至酒店列表。首页同时提供搜索历史（最多保留 10 条，去重）、热门城市快捷入口、今晚 / 明天 / 周末等日期快捷选项，以及凌晨 0–6 时的早班提示。首页「查询」按钮下方设有优惠券入口卡片，点击跳转至优惠券领取页。

8.1.3 状态变量全景

首页维护以下 React state，共同驱动搜索条件的构建与 UI 联动：

State	初始值	类型	说明
`currentTab`	`0`	`Int`	住宿类型 Tab（0=国内 / 1=海外 / 2=钟点房 / 3=民宿）
`selectedLocation`	`null`	`Object?`	已选城市对象，含 `id` 和 `name`
`searchKeyword`	`''`	`String`	关键词搜索输入框当前内容
`startDate` / `endDate`	`''`	`String`	入住 / 退房日期，格式 `YYYY-MM-DD`
`minPriceInput` / `maxPriceInput`	`''`	`String`	已确认的价格区间（提交到 URL Query）
`sliderMin` / `sliderMax`	`0` / `4500`	`Int`	价格面板内滑块当前值（确认前暂存）
`panelMinInput` / `panelMaxInput`	`''`	`String`	价格面板数字输入框内容（与滑块双向绑定）
`isPricePanelVisible`	`false`	`Bool`	价格区间底部弹出面板显示状态
`selectedTags`	`[]`	`Array`	已选标签对象数组，`id` 唯一标识
`isTagSelectorVisible`	`false`	`Bool`	标签多选弹窗显示状态
`isCitySelectorVisible`	`false`	`Bool`	城市选择器弹窗显示状态
`isCalendarVisible`	`false`	`Bool`	日历组件弹窗显示状态
`searchHistory`	`[]`	`Array<String>`	Taro Storage 持久化的搜索历史（最多 10 条）
`showSearchSuggestion`	`false`	`Bool`	输入框聚焦时展示历史+建议下拉

8.1.4 价格区间面板实现

价格区间采用底部弹出面板，包含四类交互元素，相互双向绑定：

- **数字输入框**：`<Input type='number'>` 实时校验输入值——最低价需 ⩾ `PRICE_MIN (0)` 且 < `sliderMax`，最高价需 > `sliderMin` 且 ⩽ `PRICE_MAX (4500)`；校验通过则同步更新对应滑块位置。
- **双滑块进度条**：通过 `onTouchMove` + `Taro.createSelectorQuery` 实时获取轨道 DOM 宽度并换算为价格值；两滑块通过 `zIndex` 层叠实现交叉区域的正确点击响应。
- **六个快捷标签**：「不限 / 200 以下 / 200–500 / 500–1000 / 1000–2000 / 2000 以上」点击后同时更新 `sliderMin`、`sliderMax` 和面板输入框，保持三者同步。

- **重置与确认按钮**：确认时调用 `handlePriceConfirm()`，将 `sliderMin/sliderMax` 写入外层 `minPriceInput/maxPriceInput`，搜索栏价格按钮标签由 `getPriceLabel()` 动态生成（如 `¥200-500` / `¥1000以上`）。

8.1.5 标签多选实现

标签切换逻辑由 `handleToggleTag(tag)` 实现，基于 `tag.id` 作为唯一键进行 Array 操作：

来源：`trip_front_end_taro/src/pages/home/index.jsx`

```
const handleToggleTag = (tag) => {
  setSelectedTags(prev => {
    const exists = prev.some(t => t.id === tag.id)
    return exists
      ? prev.filter(t => t.id !== tag.id)  // 已选 → 取消
      : [...prev, tag]                     // 未选 → 追加
  })
}
```

搜索时将 `selectedTags` 的 `id` 数组序列化为 URL Query（`tagIds=1,2,3`），列表页接收后执行 AND 逻辑过滤——所有选中标签均需命中才展示该酒店。

8.1.6 搜索历史实现

搜索历史通过 `Taro.getStorageSync` / `setStorageSync` 持久化到微信小程序本地存储：

- **加载**：组件挂载时 `Taro.getStorageSync('searchHistory')` 读取数组并 `setSearchHistory()`
- **保存**：每次搜索调用 `saveSearchHistory(keyword)`——先过滤重复项（`filter`），再 `unshift` 插入头部，超过 10 条时 `slice(0, 10)` 截断
- **清空**：`Taro.showModal` 二次确认后调用 `Taro.removeStorageSync('searchHistory')` 并清空 state
- **点击历史条目**：直接将该关键词填入搜索框并触发搜索

页面挂载时并发请求 `getLocations()` 和 `getTags()`，日历组件根据当前 Tab 自动切换区间模式（普通住宿）与单日模式（钟点房）。点击搜索时将所有搜索参数序列化为 URL Query，经 `Taro.navigateTo` 传至列表页。

8.1.7 城市搜索交互流程

首页和列表页均提供城市搜索功能，共用同一套交互逻辑与 A-Z 拼音索引选择器。

A-Z 拼音分组机制

自建工具模块 `trip_front_end_taro/src/utils/pinyinMap.js`，维护 `CHAR_TO_INITIAL` 映射表（覆盖 70+ 常用汉字首字 → 大写拼音字母），`getInitial(name)` 取城市名首字查表返回字母（英文直接 `toUpperCase`，未匹配返回 `#`）。`groupByInitial(cities)` 遍历城市列表按首字母分桶，再按字母排序输出 `[{letter, cities}]` 数组，供前端渲染分组视图。

城市选择器 UI

弹出层顶部提供"国内/海外"双 Tab 切换（`cityTab`），根据 `location.type` 过滤城市；下方设搜索框实时前端过滤（`name.includes(keyword)`）。城市以 3 列网格按字母分组展示，每组

显示字母标题行。弹窗右侧渲染字母索引栏，点击字母通过 `ScrollView.scrollIntoView` 快速滚动至对应分组，实现通讯录式检索体验。左上角“重置”按钮清除城市选择。

搜索条件互斥逻辑

关键词搜索与城市筛选设计为互斥关系，避免产生冲突的查询结果：

- **输入关键词** → 自动清除已选城市（`setSelectedLocation(null)`），移除 `locationId` 参数，以关键词全局搜索
- **选择城市** → 自动清空搜索关键词（`setLocalSearchKeyword('')`），移除 `keyword` 参数，按城市定向筛选
- **清空搜索框** → 300ms 防抖后自动移除 `keyword` 并重新加载全部结果，无需手动按回车
- **重置城市** → 同时清空关键词和城市，恢复无筛选状态

首页与列表页通过 `Taro.setStorageSync('selectedCitySync')` 同步城市选择状态，保证跨页面一致性。

Figure 8: 城市搜索交互流程图（含 A-Z 索引与互斥逻辑）

8.1.8 Banner 广告轮播与快速标签

Banner 轮播

首页顶部使用 Taro `<Swiper>` 组件实现广告轮播，`autoplay/circular/lazyLoad` 三属性启用自动播放、无缝循环和懒加载；每个 `SwiperItem` 绑定 `onClick` 事件，点击后通过 `Taro.navigateTo` 直接跳转至对应酒店详情页：

来源：`trip_front_end_taro/src/pages/home/index.jsx`

```
{BANNER_IMAGES.map((item, index) => (
  <SwiperItem
    key={index}
    onClick={() => Taro.navigateTo({
      url: `/pages/hotelDetail/index?id=${item.hotelId}`
```

```
    })}
  >
    <Image className='banner-img' src={item.src} mode='aspectFill' lazyLoad />
  </SwiperItem>
))}
```

Banner 图片配置集中在 `trip_front_end_taro/src/config/images.js` 的 `BANNER_IMAGES` 数组（`{src, hotelId}` 对象），便于运营人员维护。

住宿类型快速标签（Tab）

首页提供四种住宿类型 Tab：国内 / 海外 / 钟点房 / 民宿（`currentTab` 0–3）。切换 Tab 联动以下行为：

- 钟点房（Tab=2）时，Calendar 组件自动切换为**单日模式**（仅选入住日，无离店日）；
- 其余 Tab 保持**区间模式**（选入住 + 离店日期，计算住宿晚数）；
- 搜索时将 Tab 类型（`type: hotel/overseas/hourly/homestay`）序列化至 URL Query，列表页据此过滤酒店类型。

FilterPanel 星级筛选

筛选面板（`trip_front_end_taro/src/components/FilterPanel/index.jsx`）提供四维筛选：酒店星级（3 星及以上 / 4 星及以上 / 5 星）、价格区间、评分分数、酒店设施。选中的星级通过 `minStars` 回调至列表页，以 `hotel.starRating >= minStars` 进行客户端精确过滤。

8.2 酒店详情页实现

8.2.1 相关文件

- 详情页（759 行）：`trip_front_end_taro/src/pages/hotelDetail/index.jsx`
- 预订确认弹窗：`trip_front_end_taro/src/components/BookingConfirm/index.jsx`
- 数据接口：`trip_front_end_taro/src/services/hotel.js`、`trip_front_end_taro/src/services/booking.js`、`trip_front_end_taro/src/services/favorite.js`

8.2.2 功能概述

详情页展示酒店完整信息（图片画廊、设施图标、评分与评价数），列出各房型的实时价格和剩余间数，用户选择房型与入离日期后点击预订，弹出确认弹窗填写入住人信息并提交订单。页面提供收藏按钮（心形图标）和“在地图上查看”入口。

8.2.3 并行化数据加载优化

详情页需加载四类数据：酒店基本信息、房型库存价格、收藏状态、评价列表。早期实现中四个请求串行执行，总耗时约 800ms（每个请求约 200ms）。优化后通过 `Promise.all` 并行发起，总耗时降至约 250ms（取决于最慢的单个请求）：

来源：`trip_front_end_taro/src/pages/hotelDetail/index.jsx`

```
const [hotelRes, roomTypesRes, favoriteRes, commentList] = await Promise.all([
  getHotelById(hotelId),                        // 酒店详情
  getHotelRoomTypes(hotelId, startDate, endDate),  // 房型与库存
  storage.isAuthenticated()                     // 收藏状态（仅登录用户）
    ? checkFavorite(hotelId).catch(() => null)
```

```
    : Promise.resolve(null),
  getHotelCommentsCombined(hotelId),          // 评价列表
]);
```

此优化直接缩短首屏可交互时间（TTI）约 70%，提升用户感知速度，同时通过 `.catch(() => null)` 确保单个接口失败不影响其他数据展示（收藏状态查询失败时降级为“未收藏”）。

8.2.4 滚动吸顶动画

页面顶部 Header 默认全透明（背景和文字均透明），随页面向下滚动逐渐过渡为白色不透明导航栏，酒店名称同步淡入：

来源：trip_front_end_taro/src/pages/hotelDetail/index.jsx

```
const handleScroll = (e) => {
  const scrollTop = e.detail.scrollTop
  // 0–150px 线性插值，clamp 到 [0, 1]
  const opacity = Math.min(scrollTop / 150, 1)
  setHeaderOpacity(opacity)
  // 100px 阈值时显示酒店名称
  setShowHeaderTitle(scrollTop > 100)
}
```

Header 样式动态绑定：`backgroundColor: rgba(255,255,255,${headerOpacity})`，文字颜色根据 `opacity > 0.5` 在白色/深灰间切换，实现视觉上的平滑过渡。

8.2.5 房间可用性实时刷新

用户更改入离日期后，系统自动重新请求该日期段的房型库存与动态价格：

来源：trip_front_end_taro/src/pages/hotelDetail/index.jsx

```
useEffect(() => {
  if (hotelId && startDate && endDate) {
    refreshRoomAvailability(startDate, endDate)
  }
}, [startDate, endDate])  // 日期变化触发

const refreshRoomAvailability = async (start, end) => {
  const res = await getHotelRoomTypes(hotelId, start, end)
  setRoomTypes(prev => prev.map(room => {
    const updated = res.data.find(r => r.id === room.id)
    return updated
      ? { ...room, remainingRooms: updated.remainingRooms, dynamicPrice:
updated.dynamicPrice }
      : room
  }))
}
```

房型卡片优先展示 `dynamicPrice`，回退至 `RoomType.price`（基础价）；底部粘性栏实时计算总价 = 日均动态价 × 入住晚数。

8.2.6 预订确认流程

预订流程经过三重校验再提交，防止无效请求：

1. **鉴权检查**：`storage.isAuthenticated()` 为 false → `Taro.showToast` 提示并跳转登录页
2. **房型校验**：`selectedRoomTypeId` 为空 → 提示"请先选择房型"
3. **弹窗确认**：展示预订详情（酒店名、房型、日期、价格），用户填写入住人姓名、手机、到店时间，提交前校验必填项

确认提交后调用 `createBooking(bookingData)` 发送 `POST /api/bookings`，成功后跳转 `/pages/orderList/index`。

8.2.7 房型价格排序与日历晚数标注

房型按价格从低到高排序

详情页严格按照题目要求，始终将房型列表以价格从低到高排序展示。日期更改时调用 `refreshRoomAvailability` 获取动态价格后，立即对列表重新排序：

来源：`trip_front_end_taro/src/pages/hotelDetail/index.jsx`

```
updated.sort(
  (a, b) => (a.dynamicPrice ?? a.price) - (b.dynamicPrice ?? b.price)
);
```

排序优先使用当前日期段的动态价格 `dynamicPrice`，若无则回退至房型基础价 `price`，保证价格显示始终真实反映所选日期的实际报价。

日历选择与晚数粘性底栏

详情页底部有粘性悬浮栏，实时展示已选日期段及总晚数信息：

- 点击日期区域唤出 Calendar 组件，支持重新选择入住/离店日期；
- 日期变化后 `refreshRoomAvailability` 自动触发，房型价格和库存实时更新；
- 底部栏根据入住晚数和所选房型动态价格计算并展示总价（`日均价 × 晚数`）；
- 满足题目"日历 + 入住晚数展示"功能要求。

返回导航

页面顶部 Header 提供返回按钮，调用 `Taro.navigateBack()` 返回列表页，保持浏览上下文和搜索参数不变。

8.2.8 收藏切换

收藏状态由 `isFavorite` boolean 驱动，通过 `Icon` 组件在 `heart`（未收藏空心）/ `heartFill`（已收藏实心）间切换，颜色跟随收藏状态动态变化：

来源：`trip_front_end_taro/src/pages/hotelDetail/index.jsx`

```
const handleCollect = async () => {
  const res = isFavorite
    ? await removeFavorite(hotelId)   // DELETE /api/favorites
    : await addFavorite(hotelId)      // POST /api/favorites
  if (res.success) setIsFavorite(!isFavorite)
}
```

特殊情况处理：① 未登录用户点击收藏时，弹出 toast 提示“请先登录”，1.5s 后跳转登录页，不发起接口请求；② `favoriteLoading` 状态锁在请求期间阻止重复点击，`finally` 块确保锁必然释放；③ 页面初始化检查收藏状态时，未登录则跳过请求，避免 401 触发重定向；④ 后端 upsert 幂等写入，重复收藏请求不报错，UI 状态始终与数据库一致。

8.3 日历选择组件实现

8.3.1 相关文件

- 组件（430 行）：`trip_front_end_taro/src/components/Calendar/index.jsx#L16`（mode 属性定义：‘range’ 或 ‘single’）、`trip_front_end_taro/src/components/Calendar/index.jsx#L124-L156`（mode 模式切换逻辑）
- 样式（386 行）：`trip_front_end_taro/src/components/Calendar/index.css`
- 调用方：`trip_front_end_taro/src/pages/home/index.jsx`（首页日期选择）、`trip_front_end_taro/src/pages/hotelDetail/index.jsx`（详情页日期修改）
- 依赖库：dayjs（日期计算与格式化）

8.3.2 功能概述

自研日历组件，支持单日选择（钟点房模式）和日期区间选择（普通住宿模式）。用月份网格形式渲染，支持向前/向后翻月，今天之前及 30 天之后的日期不可选。底部固定确认栏始终可见，用户选好日期后需点击“确定”按钮才会提交（不自动关闭），防止误操作。

8.3.3 月份网格生成算法

每月的日历以 $6 \times 7 = 42$ 个格子渲染，通过 `generateMonthGrid()` 函数计算：

来源：`trip_front_end_taro/src/components/Calendar/index.jsx`

```
const generateMonthGrid = () => {
  const grid = []
  const firstDayOfWeek = monthStart.day()  // 0=日 1=一 … 6=六

  // 1. 前置空格：当月第一天前的空位
  for (let i = 0; i < firstDayOfWeek; i++) grid.push(null)

  // 2. 填充当月所有日期
  let d = monthStart.clone()
  while (d.isBefore(monthEnd) || d.isSame(monthEnd, 'day')) {
    grid.push(d.clone())
    d = d.add(1, 'day')
  }

  // 3. 后置空格：补齐至 42 格
  while (grid.length < 42) grid.push(null)
  return grid
}
```

`null` 格渲染为空白占位，`dayjs` 对象格渲染为日期数字。

8.3.4 模式切换核心逻辑

日历通过 mode 属性（trip_front_end_taro/src/components/Calendar/index.jsx#L16）支持两种选择模式，handleDayClick 函数（trip_front_end_taro/src/components/Calendar/index.jsx#L124-L156）根据模式走不同分支：

来源：trip_front_end_taro/src/components/Calendar/index.jsx#L124-L156

```
if (mode === 'single') {
  // 单日期模式（钟点房）：选中后等待用户点"确定"
  setTempStart(dayStr)
  setTempEnd('')
} else {
  // 区间模式（普通住宿）
  if (!tempStart || tempEnd) {
    // 首次点击 or 已选完区间重新选
    setTempStart(dayStr)
    setTempEnd('')
  } else {
    if (dayStr === tempStart) return  // 不允许零晚预订
    // 自动确定入住/离店顺序
    const firstDay = dayjs(tempStart)
    if (secondDay.isAfter(firstDay, 'day')) {
      setTempStart(tempStart); setTempEnd(dayStr)
    } else {
      setTempStart(dayStr); setTempEnd(tempStart)  // 自动交换
    }
  }
}
```

首页住宿类型 Tab 切换时自动设置日历模式：钟点房 Tab 传入 mode='single'，其余 Tab 传入 mode='range'，实现 Tab 联动日历行为。

8.3.5 日期范围选择状态机

区间模式下，日期选择经历三个阶段，由 handleDayClick 驱动，内部维护 tempStart / tempEnd 两个状态：

当前状态	用户操作	结果
空（未选）或区间已完成	点击任意日期	tempStart = 所选日，tempEnd = ''，重新开始选择
已有 tempStart	点击晚于起点的日期	tempEnd = 所选日，区间确定，底部确认栏显示“共 N 晚”摘要
已有 tempStart	点击早于起点的日期	自动交换顺序：tempStart = 所选日 / tempEnd = 原起点
已有 tempStart	点击与起点相同日期	忽略（不允许零晚预订）
区间已完成	点击“确定”按钮	调用 onSelect(start, end) + onConfirm() + 关闭日历
区间已完成	重新点击任意日期	清空区间，重新开始选择（不关闭日历）
任意状态	翻页切换月份	保留已选日期，允许跨月选择区间

单日模式（钟点房）下，点击任意日期后 tempStart 高亮，用户需点击“确定”按钮才提交并关闭。

8.3.6 底部确认栏与布局优化

日历采用 Flex 纵向布局 + 可滚动内容区设计，保证底部确认栏在任何屏幕高度下始终可见：

- `.calendar-main`：`max-height: 80vh; display: flex; flex-direction: column;`
- `.calendar-header` / `.month-switcher` / `.calendar-footer`：`flex-shrink: 0`（固定不收缩）
- `.calendar-scroll-body`：`flex: 1; overflow-y: auto;`（日期网格区域可滚动）

确认按钮采用高级感胶囊设计：`width: 240rpx; height: 88rpx; border-radius: 44rpx;` 圆角胶囊造型，未激活态为灰色（`var(--color-bg-tertiary)`），选好日期后渐变蓝（`linear-gradient(135deg, #1677ff, #4a90e2)`）+ 双层投影 + `cubic-bezier` 缓动过渡，按压时 `scale(0.96)` 反馈。`env(safe-area-inset-bottom)` 适配全面屏安全区。

8.3.7 禁用日期处理

来源：`trip_front_end_taro/src/components/Calendar/index.jsx`

```
const isDaySelectable = (day) => {
  if (!day) return false
  // 不可选：今天之前 / 30天之后
  return !day.isBefore(today, 'day') && !day.isAfter(maxDate, 'day')
}
```

不可选日期添加 `disabled` CSS class，`onClick` 事件短路返回，视觉上以低透明度显示区分可选状态。已选区间内的日期添加 `in-range` class 展示蓝色背景条带。

8.4 酒店列表与筛选实现

8.4.1 相关文件

- 列表页：`trip_front_end_taro/src/pages/hotelList/index.jsx`
- 筛选面板：`trip_front_end_taro/src/components/FilterPanel/index.jsx`
- 数据接口：`trip_front_end_taro/src/services/hotel.js`（`getHotels`）

8.4.2 筛选头部栏（四元素条形导航）

列表页顶部固定筛选头部栏，包含题目要求的四个核心元素，满足“核心条件筛选头部”功能要求：

元素	交互行为
城市名（`selectedLocation.name`）	点击打开城市选择弹窗，选中后立即以新城市刷新列表
入住/离店日期	点击唤出 Calendar 组件，确认后更新 `startDate`/`endDate` 并重新加载
入住晚数（`getNightCount()`）	根据当前日期自动计算并展示“共 N 晚”，日期变化实时更新
筛选设置图标	点击展开 FilterPanel（星级/价格/评分/设施四维筛选），有活跃筛选时显示红点徽标

`getNightCount()` 的计算逻辑：

来源：`trip_front_end_taro/src/pages/hotelList/index.jsx`

```
const getNightCount = () => {
  const start = startDate ? dayjs(startDate) : today;
  const end = endDate ? dayjs(endDate) : tomorrow;
  if (end.isAfter(start, 'day')) {
```

```
    return `共${end.diff(start, 'day')}晚`;
  }
  return '共1晚';
};
```

酒店卡片信息：每张酒店卡片展示酒店中文名（nameZh）、用户综合评分（score，由历史评价汇总）、地址、以及从后端返回的 roomTypes 数组计算得出的最低房型价格，满足题目“每条酒店可展示名称/评分/地址/价格”的要求。

N+1 查询优化：早期实现中，列表页对每个酒店单独调用 getHotelMinPrice(hotelId) 获取最低价，导致 10 条列表产生 11 次 SQL 查询（1 次查酒店 + 10 次查价格）。优化后，后端 GET /api/hotels 通过 Prisma include: { roomTypes: { select: { price: true } } } 一次性返回全部房型价格，前端直接在内存中计算 Math.min(...roomTypes.map(rt => rt.price))，查询次数降至 1 次，响应延迟从约 500ms 降至约 150ms（10 条数据）。

来源：trip_front_end_taro/src/pages/hotelList/index.jsx

```
// 直接从后端已返回的 roomTypes 计算最低价，避免 N+1 请求
const prices = (hotel.roomTypes || [])
  .map(rt => parseFloat(rt.price))
  .filter(p => p > 0);
const minPrice = prices.length > 0 ? Math.min(...prices) : 0;
```

8.4.3 做了什么

列表页展示符合搜索条件的酒店卡片，支持按推荐 / 价格升降序排序，以及价格区间 / 星级 / 设施多维度筛选；支持上滑自动加载更多（无限滚动分页）；可一键切换为地图视图，在地图上以价格气泡标注各酒店位置，点击气泡显示酒店缩略卡片。

8.4.4 怎么做的

上滑自动加载（长列表渲染优化）

列表体使用 Taro <ScrollView> 包裹，距底部 100px 时触发 loadMoreHotels 回调：

来源：trip_front_end_taro/src/pages/hotelList/index.jsx

```
<ScrollView
  scrollY
  onScrollToLower={loadMoreHotels}
  lowerThreshold={100}
>
  {/* 酒店卡片列表 */}
</ScrollView>
```

loadMoreHotels 通过 loadingMore 布尔互斥锁防止重复触发，hasMore 状态标识是否还有更多数据：

```
const loadMoreHotels = async () => {
  if (loadingMore || !hasMore) return;   // 互斥锁 + 终止条件
  setLoadingMore(true);
  const nextPage = currentPage + 1;
```

```
  // 请求下一页数据，追加至 hotelList ...
};
```

后端 GET /api/hotels?page=N&limit=10 返回 { data, total } 字段，前端以 已加载数 >= total 计算 hasMore，首屏仅渲染 10 条 DOM 节点，随滚动追加而非替换，内存占用与数据量线性正相关，满足题目“酒店列表需支持上滑自动加载功能”和技术复杂度“长列表渲染优化（3 分）”要求。

排序与筛选

页面加载时将首页传入的 URL 参数解码为 searchParams 对象，解码过程用 try/catch 包裹，格式异常时安全回退为空对象，防止整页崩溃。酒店数据格式化时，hotel.images 和 hotel.facilities 的 JSON.parse 均以 try/catch 包裹，解析失败时回退为空数组，保证即使后端返回格式异常的 JSON 字段也不影响列表渲染。所有后续的排序、筛选、关键词二次搜索均在客户端 filterAndSortHotels() 函数内完成，避免重复请求：关键词对酒店名称 / 描述 / 标签做大小写不敏感模糊匹配；价格区间和星级做精确过滤；设施标签要求全部命中（AND 逻辑）。FilterPanel 组件以受控方式接收 defaultFilters 并通过 onConfirm 回调将新筛选值传回列表页触发重算。地图模式下将酒店坐标转换为 Taro <Map> 的 markers 数组，callout.content 显示价格，点击标记更新 selectedHotel 展示详情卡。

8.5 订单流程实现

8.5.1 相关文件

- 订单列表：trip_front_end_taro/src/pages/orderList/index.jsx
- 订单详情：trip_front_end_taro/src/pages/orderDetail/index.jsx
- 提交评价：trip_front_end_taro/src/pages/submitReview/index.jsx
- 后端接口：app/api/bookings/[id]/route.tsx（GET / PUT / DELETE）

8.5.2 做了什么

订单列表页按状态分 Tab 展示用户的全部预订（待确认 / 已确认 / 已完成 / 已取消）。订单详情页展示入住日期、入住人、房型、总价及优惠折扣，状态为 pending / confirmed 时提供“取消订单”按钮；状态为 completed 且尚未评价时提供“提交评价”入口。提交评价页支持 1–5 星点选和文字输入，完成后返回评价列表。

8.5.3 怎么做的

取消订单——库存归还事务

来源：app/api/bookings/[id]/route.tsx

```
// PUT /api/bookings/[id]  body: { status: 'cancelled' }
await prisma.$transaction(async (tx) => {
  await tx.booking.update({ where: { id: bookingId }, data: { status:
'cancelled' } });
  // 逐日归还库存
  let cur = new Date(booking.checkInDate);
  while (cur < new Date(booking.checkOutDate)) {
    await tx.roomAvailability.updateMany({
      where: { roomTypeId: booking.roomTypeId, date: cur },
```

```
      data: { booked: { decrement: 1 } },
    });
    cur.setDate(cur.getDate() + 1);
  }
});
```

取消与归还在同一个 Prisma 事务内完成，任一步骤失败均会回滚，保证库存数据一致性。商户和订单归属用户均可发起取消。前端通过 cancellingId 状态锁防止同一订单被重复取消（按钮点击后立即锁定，finally 块释放），并额外保存 hotelId 字段，供“去评价”按钮跳转 submitReview 页时传递，使评价与对应酒店正确关联。

提交评价——星评交互

来源：trip_front_end_taro/src/pages/submitReview/index.jsx

```
// submitReview 页: 5 颗星点选
{[1,2,3,4,5].map(star => (
  <Text
    key={star}
    className={`star-icon ${rating >= star ? 'active' : ''}`}
    onClick={() => setRating(star)}
  >★</Text>
))}
```

rating >= star 使已选评分之前的所有星变为高亮色。提交时调用 createReview(hotelId, rating, content)，通过 /api/comments 接口将评价写入对应酒店；后端以 bookingId @unique 约束防止重复评价，重复提交返回 Prisma P2002 错误。

8.6 收藏列表页实现

8.6.1 相关文件

- trip_front_end_taro/src/pages/favoriteList/index.jsx
- trip_front_end_taro/src/services/favorite.js（getFavorites）

8.6.2 做了什么

展示当前用户收藏的所有酒店卡片列表，支持取消收藏操作，点击卡片跳转酒店详情页。

8.6.3 怎么做的

页面挂载时并发调用 getFavoriteFolders() 和 getMyFavorites()（trip_front_end_taro/src/pages/favoriteList/index.jsx#L35-L38），支持收藏夹分组管理。取消收藏通过二次确认弹窗 + removeFavorite(hotelId) 发送 DELETE /api/favorites/{hotelId}，成功后乐观更新本地 state（trip_front_end_taro/src/pages/favoriteList/index.jsx#L108-L130）。空列表展示 EmptyState，支持下拉刷新（usePullDownRefresh）和批量操作模式。

来源：trip_front_end_taro/src/pages/favoriteList/index.jsx#L108-L130

```
// 取消收藏——二次确认 + 乐观更新
const handleRemoveFavorite = (hotelId, e) => {
  e.stopPropagation();
  Taro.showModal({
```

```
    title: '取消收藏', content: '确定要取消收藏这家酒店吗？',
    success: async (res) => {
      if (res.confirm) {
        const result = await removeFavorite(hotelId);
        if (result.success) {
          Taro.showToast({ title: '已取消收藏', icon: 'success' });
          setFavorites(favorites.filter(f => f.hotelId !== hotelId));
        }
      }
    }
  });
};
```

8.7 我的评价列表页实现

8.7.1 相关文件

- trip_front_end_taro/src/pages/reviewList/index.jsx
- trip_front_end_taro/src/services/review.js（getMyReviews）

8.7.2 做了什么

展示当前用户提交的所有酒店评价，每条评价显示酒店名称、评分星级、评价内容、提交时间，支持删除评价操作。

8.7.3 怎么做的

页面挂载前先通过 storage.isAuthenticated() 校验登录态，未登录自动跳转登录页（trip_front_end_taro/src/pages/reviewList/index.jsx#L21-L26）。评价卡片展示 '★'.repeat(rating) + '☆'.repeat(5-rating) 五星评分，支持行内编辑（trip_front_end_taro/src/pages/reviewList/index.jsx#L48-L58 handleEdit 切换编辑态）和删除操作。删除通过二次确认弹窗 + deleteReview(reviewId) 乐观更新列表。

来源：trip_front_end_taro/src/pages/reviewList/index.jsx#L77-L93

```
// 删除评价——二次确认 + 乐观更新
const handleDelete = (reviewId) => {
  Taro.showModal({
    title: '提示', content: '确定要删除这条评价吗？',
    success: async (res) => {
      if (res.confirm) {
        await deleteReview(reviewId);
        setReviews(prev => prev.filter(r => r.id !== reviewId));
        Taro.showToast({ title: '删除成功', icon: 'success' });
      }
    }
  });
};
```

8.8 优惠券页实现

8.8.1 相关文件

- 优惠券页：`trip_front_end_taro/src/pages/Coupon/index.jsx`
- 数据接口：`trip_front_end_taro/src/services/coupon.js`（`getCoupons`、`getUserCoupons`、`claimCoupon`）
- 后端接口：`app/api/coupons/[id]/claim/route.ts#L37-L60`（积分兑换事务：检查余额 → 扣减积分 → 发放优惠券）、`app/api/user/coupons/route.ts`

8.8.2 做了什么

实现完整的优惠券积分兑换体系，包括：① 优惠券展示与领取/兑换；② 预订时优惠券选择与抵扣；③ 积分累积机制；④ 优惠券生命周期管理。

业务流程：

- **积分来源**：用户完成订单后自动获得积分奖励（当前规则：每消费 1 元获得 1 积分，在 `Booking.status` 变更为 `completed` 时累加到 `User.points`）
- **领取/兑换**：优惠券页展示所有可用券，免费券（points = 0）直接领取，积分券（points > 0）需扣减用户积分
- **使用抵扣**：预订确认弹窗加载用户已领取且未使用的优惠券（`isUsed = false` + `validFrom ≤ now ≤ validTo` + `minSpend ≤ totalPrice`），用户手动选择一张券，前端计算优惠后金额展示
- **生命周期**：过期优惠券（`now > validTo`）前端标记为"已失效"且不可领取，不自动删除（保留用户领取记录便于追溯）

8.8.3 怎么做的

页面挂载时通过 `Promise.allSettled` 并发调用 `getCoupons()`（全量优惠券）和 `getUserCoupons()`（当前用户已领取记录），将已领取券的 ID 构建为 `Set<claimedIds>`，渲染时以 O(1) 查询判断每张券是否已领取。

点击领取时，前端首先通过 `storage.isAuthenticated()` 检查登录态，未登录则弹出 `Taro.showToast` 提示"请先登录后领取"并直接返回，不发起请求。已登录则调用 `claimCoupon(couponId)` 发送 `POST /api/coupons/{id}/claim`；请求期间按钮切换为加载态（`coupon-btn-loading`）防止重复点击；成功后将该 ID 追加进 `claimedIds` Set 并触发重新渲染，按钮变为已领取态（`coupon-btn-claimed`）。

积分兑换事务处理：后端服务端校验优惠券有效期后，若该券 `points > 0`，则启动 `prisma.$transaction` 事务（`app/api/coupons/[id]/claim/route.ts#L37-L60`）：① 查询用户当前积分余额，积分不足则抛出 `INSUFFICIENT_POINTS` 异常并返回 400；② 通过 `user.update({ data: { points: { decrement: coupon.points } } })` 原子扣减积分；③ 写入 `UserCoupon` 关联表发放优惠券。三步操作在同一事务内原子完成，保证积分扣减与优惠券发放的强一致性。若 `points = 0`，则直接写入 `UserCoupon` 表，Prisma 复合主键 `@@id([userId, couponId])` 保证幂等，重复领取返回 400。

8.8.4 积分兑换事务流程

当用户点击“积分兑换”按钮领取优惠券时，系统通过数据库事务（`app/api/coupons/[id]/claim/route.ts#L37-L60`）保证积分扣减与优惠券发放的原子性：

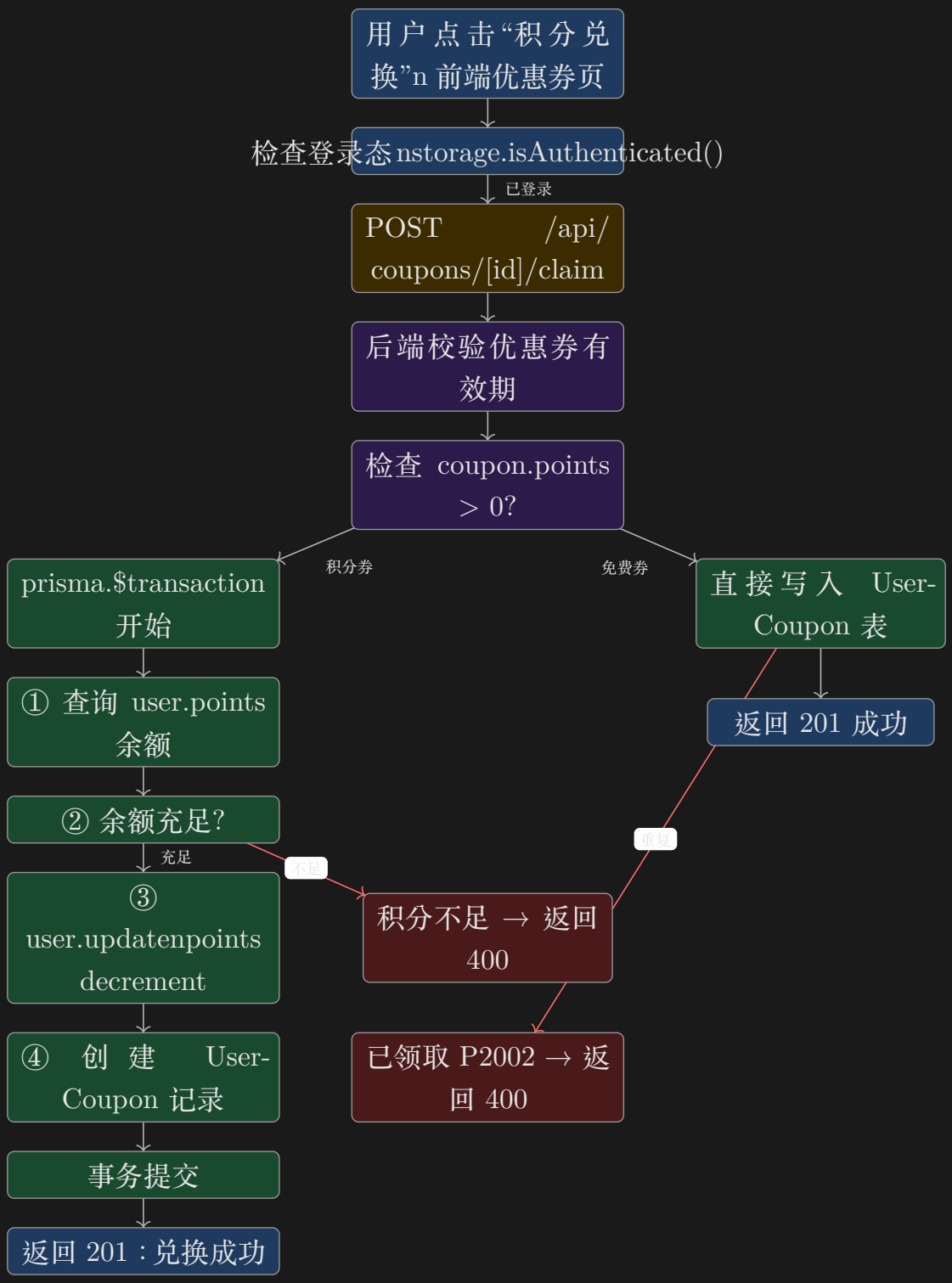

Figure 9: 积分兑换优惠券事务流程图

8.9 我的优惠券页实现

8.9.1 相关文件

- 页面：`trip_front_end_taro/src/pages/myCoupons/index.jsx`
- 数据接口：`trip_front_end_taro/src/services/coupon.js`（`getUserCoupons`）

8.9.2 做了什么

展示当前用户已领取的全部优惠券，每张卡片显示名称、描述、折扣金额、最低消费门槛和有效期，按状态区分**可用**、**已使用**和**已过期**三种样式。可用券提供“去使用”按钮跳转酒店列表页。

8.9.3 怎么做的

页面挂载时调用 `getUserCoupons()` 获取用户优惠券列表（返回 `UserCoupon` 记录，内联 `coupon` 对象）。每张卡片通过两个条件判断状态：`dayjs(item.validTo).isBefore(dayjs())` 判断是否过期，`userCoupon.status === 'USED'` 判断是否已使用。三种状态对应不同样式——可用券为正常配色 + 蓝色“去使用”按钮（点击后 `Taro.switchTab` 跳转酒店列表页），已过期和已使用券整体灰化（`coupon-card-expired` class）+ 禁用态按钮文字。卡片布局采用左右分栏：左侧展示名称、描述、标签（优惠券/已过期/已使用）和有效期，右侧展示折扣金额和消费门槛。空列表展示“暂无优惠券”提示。

8.10 地图导航实现

8.10.1 相关文件

- 地图页面：`trip_front_end_taro/src/pages/hotelMap/index.jsx`
- 酒店详情入口：`trip_front_end_taro/src/pages/hotelDetail/index.jsx`（“在地图上查看”按钮）
- 数据来源：Hotel 表 `latitude` / `longitude` 字段（GCJ-02 国测局坐标系）

8.10.2 做了什么

用户在酒店详情页点击“地图”按钮后，跳转至 `hotelMap` 页面：同时显示酒店位置（绿色 callout 标记，含酒店名称）和用户当前定位（蓝色标记），并提供“导航到此”按钮，点击后调起微信内置地图导航。

8.10.3 怎么做的

`hotelMap` 页面使用 Taro `<Map>` 组件，支持“单酒店详情”和“搜索列表”两种模式（通过路由参数 `hotelId` 区分）。组件挂载时调用 `Taro.getLocation({ type: 'gcj02' })`（`trip_front_end_taro/src/pages/hotelMap/index.jsx#L41-L51`）获取用户坐标。单酒店模式直接以传入坐标渲染 marker；列表模式通过 `getHotels()` 加载多酒店并以价格气泡标注。“导航”按钮触发 `Taro.openLocation()` 拉起微信内置地图导航，坐标统一 GCJ-02 无需转换。

来源：`trip_front_end_taro/src/pages/hotelMap/index.jsx#L147-L165`

```
// 导航到酒店——调起微信内置地图
const handleNavigate = () => {
  const lat = singleMode ? singleLat : selectedHotel?.latitude;
  const lng = singleMode ? singleLng : selectedHotel?.longitude;
  if (!lat || !lng) {
    Taro.showToast({ title: '该酒店暂无坐标信息', icon: 'none' });
    return;
  }
```

```
  Taro.openLocation({ latitude: lat, longitude: lng, name, address, scale: 16 });
};
```

9 接口文档

9.1 接口约定

约定项	说明
Base URL	`http://localhost:3000/api`（开发环境）
认证方式	HTTP Header：`Authorization: Bearer <accessToken>`
成功响应	`{ "success": true, "data": {...} }` 或 `{ "success": true, "data": [...] }`
错误响应	`{ "success": false, "error": "<描述>" }`，HTTP 状态码 400 / 401 / 403 / 404 / 500
Token 规则	Access Token 有效期 1h；过期后以 Refresh Token（7d）调用 `/auth/refresh` 换取新 Token，无感刷新
权限标注	`公开`=无需登录；`AUTH`=需登录（任意角色）；`MERCHANT`=商户角色；`ADMIN`=管理员角色
Swagger 文档	`http://localhost:3000/api-doc`（基于 JSDoc 注释自动生成 OpenAPI 3.0 规范）
日期格式	请求体中日期字段均使用 ISO 8601 字符串（`YYYY-MM-DD` 或 `YYYY-MM-DDTHH:mm:ssZ`）

9.2 认证接口 `/api/auth`

方法	路径	权限	请求体字段	响应说明
POST	`/auth/login`	公开	`email`（必填） `password`（必填）	返回 `{ accessToken, refreshToken, user: {id,email,name,role} }`；用户不存在或密码错误返回 401

POST	`/auth/register`	公开	`email`（必填）
			`password`（必填）
			`name?`
			`phone?`
			`role?`（“user”|“merchant”，默认“user”）
				返回 `{ success, data: User }`；邮箱已存在返回 400；密码经 bcrypt salt=10 哈希后存储

POST	`/auth/refresh`	公开	`refreshToken`（必填）	返回新 `accessToken`；Refresh Token 过期/无效返回 401

9.3 用户接口 `/api/users`

方法	路径	权限	参数	说明
GET	`/users`	ADMIN	—	所有用户列表，含角色信息
GET	`/users?myStaff=true`	MERCHANT	—	商户查询自己的职员列表（`merchantId = 当前用户 ID`）
GET	`/users?role=X`	公开	Query: `role`	按角色筛选用户基本信息（id/name/email），用于注册页选择商户
GET	`/users/profile`	AUTH	—	当前登录用户资料，包含 `id/email/name/phone/role/points` 等字段；由 `verifyAuth()` 保护
PUT	`/users/[id]`	AUTH	Body: `name? phone? oldPassword? newPassword?`	更新用户信息或修改密码；需为本人、ADMIN 或商户管理自己的职员（`targetUser.merchantId === currentUserId`）；密码修改仅限本人，需 `bcrypt.compare` 验证旧密码
DELETE	`/users/[id]`	AUTH	—	删除用户，需为本人、ADMIN 或商户删除自己的职员

方法	路径	权限	参数	说明
GET	`/users/[id]/bookings`	AUTH	—	该用户全量预订记录（管理员查询用）
GET	`/users/[id]/favorite`	AUTH	—	该用户收藏酒店列表

9.4 酒店接口 `/api/hotels`

方法	路径	权限	参数	说明
GET	`/hotels`	公开	Query: `locationId?` `status?`（默认 published）`tags?`（逗号分隔 ID，AND 匹配）`keyword?`（模糊匹配名称/地址）`merchantId?` `page?`（默认 1）`limit?`（默认 10，最大 100）	返回含 location / hotelTags / roomTypes 的酒店数组，并附带 `total`、`page`、`limit` 分页字段；小程序列表页使用 `page+limit` 实现上滑无限滚动加载
POST	`/hotels`	MERCHANT	Body: `nameZh`、`address`、`merchantId`（必填）；`locationId?`、`starRating?`、`images?`、`facilities?`、`roomTypes[]?`、`tagIds[]?`	创建酒店，status 默认 pending，等待管理员审核才可被用户搜索到
GET	`/hotels/[id]`	公开	—	酒店详情，含 roomTypes / location / merchant({id,name,email}) / hotelTags
PUT	`/hotels/[id]`	本人或 ADMIN	Body: 任意 Hotel 字段；`status?`（见权限说明）；`rejectionReason?`	status 变更需 HOTEL_AUDIT 权限（offline 允许本人）；变更时事务内原子写 HotelAuditLog
DELETE	`/hotels/[id]`	本人或 ADMIN	—	本人所有者或拥有 HOTEL_DELETE 权限方可删除，删除酒店及关联数据
GET	`/hotels/[id]/room-types`	公开	—	该酒店所有房型列表
GET	`/hotels/[id]/bookings`	MERCHANT	—	该酒店全量预订列表，含 user / roomType 信息
GET	`/hotels/[id]/review`	公开	—	该酒店评价列表，含用户昵称/头像信息
GET	`/hotels/[id]/audit_log`	ADMIN	—	该酒店状态变更历史，含 operator 信息和操作时间

9.5 预订接口 `/api/bookings`

`POST /api/bookings` 是系统最复杂的接口，内部执行数据库事务（详见第 4.2 节）。任何日期售罄或关闭时返回 500 并携带具体错误信息，客户端应提示用户重新选择日期。

方法	路径	权限	参数	说明
GET	`/bookings`	AUTH	Query: `merchantId?`（商户查名下全部预订，必须等于当前用户 ID）	用户查本人预订（含 hotel / roomType 信息）；merchantId 鉴权防止枚举他人订单
POST	`/bookings`	AUTH	Body: `hotelId`、`roomTypeId`、`checkInDate`、`checkOutDate`（必填）；`guestCount?`（默认 1）、`guestInfo?`	Zod 校验 + 事务创建；售罄/关闭/券无效返回 400/500；成功返回 201 + 含 `discountAmount` 的 Booking 记录

方法	路径	权限	参数	说明
			（JSON）；`couponId?`（优惠券 ID，下单自动扣减折扣）	
GET	`/bookings/[id]`	AUTH	—	预订详情，含 hotel（含 merchantId / 经纬度）、roomType、review 信息；订单归属用户或对应商户可查
PUT	`/bookings/[id]`	AUTH	Body: `status`（状态）或 `guestInfo`（JSON）	用户或商户均可取消（同一事务归还逐日库存）；其余状态变更仅商户操作
DELETE	`/bookings/[id]`	AUTH	—	硬删除订单；`pending` / `confirmed` 状态同步归还库存

9.6 房型与库存接口 `/api/room-types`

方法	路径	权限	参数	说明
GET	`/room-types`	公开	Query: `hotelId?`	房型列表，可按酒店过滤
POST	`/room-types`	MERCHANT	Body: `hotelId`、`name`、`price`（必填）；`description?`、`stock?`（默认 10）、`discount?`（默认 1.00）、`amenities?`、`images?`	创建房型，内部验证 hotelId 属于当前商户，防止越权
GET	`/room-types/[id]`	公开	—	房型详情
PUT	`/room-types/[id]`	MERCHANT	Body: 任意房型字段	更新房型，验证归属关系
DELETE	`/room-types/[id]`	MERCHANT	—	删除房型，验证归属关系
GET	`/room-types/[id]/availability`	公开	Query: `startDate`、`endDate`	按日期范围返回 RoomAvailability 列表，前端日历组件用于渲染价格/库存
POST	`/room-types/[id]/availability`	MERCHANT	Body: `data[]`（每项含 `date`、`price?`、`quota?`、`isClosed?`）	批量 upsert 日期库存与动态价格，商户日历批量设价使用此接口

9.7 评价接口 `/api/reviews`

方法	路径	权限	参数	说明
GET	`/reviews`	AUTH	—	当前用户所有评价，含 hotel（名称/图片）/ booking（日期）信息
POST	`/reviews`	AUTH	Body: `hotelId`、`rating`（1–5 整数）、`content`（必填）	实际调用 `/api/comments` 接口，通过 `hotelId` 关联酒店；后端校验 `bookingId` `@unique` 防止一单多评
DELETE	`/reviews/[id]`	AUTH	—	仅评价本人可删除，其他用户返回 403

9.8 评论接口 `/api/comments`

评论与评价为两个独立模块：评价（Review）基于 `bookingId` 一单一评，评论（Comment）直接关联 `hotelId` 支持自由评论。创建和删除评论时自动调用 `updateHotelScore()` 重算酒店综合评分。

方法	路径	权限	参数	说明
GET	`/comments`	公开	Query: `hotelId?`、`userId?`	评论列表，含 user(id,name) 和 hotel(id,nameZh)，按创建时间降序

方法	路径	权限	参数	说明
POST	`/comments`	AUTH	Body: `hotelId`（必填）、`content`（必填）、`score?`（评分）	创建评论，自动调用 `updateHotelScore()` 更新酒店评分
GET	`/comments/[id]`	公开	—	单条评论详情，含 user 和 hotel 信息
PUT	`/comments/[id]`	AUTH	Body: `content?`、`score?`	仅评论作者可修改，修改后自动更新酒店评分
DELETE	`/comments/[id]`	AUTH	—	评论作者或 ADMIN 可删除，删除后自动更新酒店评分

9.9 收藏接口 `/api/favorites`

方法	路径	权限	参数	说明
GET	`/favorites`	AUTH	—	收藏列表含酒店完整详情；返回的 id 字段格式为 `"userId-hotelId"`（便于前端直接用作 React key）
POST	`/favorites`	AUTH	Body: `hotelId`（必填）	upsert 写入 数据库复合主键保证幂等，重复收藏不报错
DELETE	`/favorites/[id]`	AUTH	Path: id = hotelId	按 hotelId 取消当前用户的收藏
GET	`/favorites/check/[id]`	AUTH	Path: id = hotelId	返回 `{ isFavorited: boolean }`，详情页初始化心形按钮状态使用

9.10 优惠券接口 `/api/coupons`

方法	路径	权限	参数	说明
GET	`/coupons`	公开	—	返回全部优惠券列表，含 `id`、`code`、`name`、`discount`、`minSpend`、`validFrom`、`validTo`、`points`（兑换所需积分）等字段
POST	`/coupons`	ADMIN	Body: `code`、`name`、`discount`（必填）；`minSpend?`、`validFrom?`、`validTo?`、`points?`（所需积分，默认 0）	管理员创建优惠券，`code` 全局唯一；`points > 0` 表示积分兑换券
GET	`/coupons/[id]`	公开	—	单张优惠券详情
PUT	`/coupons/[id]`	ADMIN	Body: 任意优惠券字段	更新优惠券信息
DELETE	`/coupons/[id]`	ADMIN	—	删除优惠券及关联的用户领取记录
POST	`/coupons/[id]/claim`	AUTH	—	用户领取或积分兑换优惠券；服务端校验有效期（`validFrom`/`validTo`），若优惠券 `points > 0` 则启动事务：① 检查用户积分余额，不足返回 400；② 原子扣减 `user.points`；③ 写入 `UserCoupon` 发放券。免费券（`points = 0`）直接写入。重复领取返回 400（Prisma P2002）
GET	`/user/coupons`	AUTH	—	返回当前用户已领取的优惠券列表，含 `isUsed`、`usedAt` 及完整优惠券信息（内联 `coupon` 对象）

积分兑换事务核心代码：

来源：`app/api/coupons/[id]/claim/route.ts#L36-L60`

```
// 积分兑换：事务保证原子性
if (coupon.points > 0) {
  return await prisma.$transaction(async (tx) => {
    // 1. 查询用户当前积分余额
    const user = await tx.user.findUnique({
      where: { id: userId },
      select: { points: true }
    });
    if (!user || user.points < coupon.points) {
      throw new Error('INSUFFICIENT_POINTS');  // 触发事务回滚
    }
    // 2. 原子扣减积分（Prisma decrement 保证并发安全）
    await tx.user.update({
      where: { id: userId },
      data: { points: { decrement: coupon.points } }
    });
    // 3. 发放优惠券
    const userCoupon = await tx.userCoupon.create({
      data: { userId, couponId },
    });
    return NextResponse.json({ success: true, data: userCoupon }, { status:
201 });
  });
}
// 免费券：直接写入，@@id([userId, couponId]) 复合主键防重复
const userCoupon = await prisma.userCoupon.create({ data: { userId,
couponId } });
```

9.11 城市与标签接口 /api/locations 和 /api/tags

方法	路径	权限	参数	说明
GET	/locations	公开	Query: name?（城市名搜索）、type?（domestic/overseas/province）	返回城市列表，支持按名称模糊搜索和类型筛选；含 parentId 字段表示层级关系（省-市结构）
POST	/locations	ADMIN	Body: name、description?、type、parentId?	管理员创建城市/省份；type="province" 为省级，type="domestic" 为国内城市（需 parentId 关联省份），type="overseas" 为海外城市
GET	/locations/[id]	公开	—	单个城市详情，含关联的酒店数量
PUT	/locations/[id]	ADMIN	Body: 任意 location 字段	更新城市信息
DELETE	/locations/[id]	ADMIN	—	删除城市；若有酒店关联会返回 400 阻止删除
GET	/tags	公开	—	返回所有标签列表（含早餐、WiFi、停车场等）
POST	/tags	ADMIN	Body: name	管理员创建标签，name 全局唯一
PUT	/tags/[id]	ADMIN	Body: name	更新标签名称

方法	路径	权限	参数	说明
DELETE	`/tags/[id]`	ADMIN	—	删除标签；会自动清理 `hotel_tags` 关联记录

9.12 其他辅助接口

方法	路径	权限	参数	说明
GET	`/room-availability/[id]`	公开	—	查询单条库存记录（按 RoomAvailability.id），返回 `roomTypeId`、`date`、`quota`、`booked`、`price`
PUT	`/room-availability/[id]`	MERCHANT	Body: `quota?`、`price?`	商户更新单日库存配额或动态价格
GET	`/map/geocoder`	公开	Query: `address`（地址字符串）	腾讯地图地址解析接口，返回 GCJ-02 坐标 `{ lat, lng }`；用于酒店录入时根据地址自动填充经纬度
GET	`/swagger`	公开	—	返回 OpenAPI JSON 规范，供 `/api-doc` 页面的 Swagger UI 渲染使用

9.13 AI 推荐接口 `POST /api/ai/recommend`

权限：公开（无需登录，降低使用门槛）

请求体格式：发送完整对话历史，支持多轮上下文理解。

```
{
  "messages": [
    { "role": "user",      "content": "找北京带早餐的五星级酒店" },
    { "role": "assistant", "content": "为您推荐以下酒店..." },
    { "role": "user",      "content": "有没有价格更低的？" }
  ]
}
```

响应格式：`Content-Type: text/plain; charset=utf-8`（流式文本），前端通过 `ReadableStream.getReader()` 逐块读取文字片段拼接渲染。

内部处理步骤：① LangChain 结构化提取意图（最近 3 条消息，temperature=0）→ ② Prisma 查询并过滤候选酒店（最多取 20 条，客户端排序后取前 5）→ ③ 构造含酒店上下文的 System Prompt → ④ ChatOpenAI 流式生成返回（temperature=0.7）。

9.14 文件上传接口 `POST /api/upload`

权限：AUTH（需登录，防止未授权用户滥用存储）

请求：`Content-Type: multipart/form-data`，字段名 `file`

文件名安全化处理：过滤原始文件名中的非字母数字字符，拼接时间戳与随机串，防路径穿越攻击：

来源：`app/api/upload/route.ts`

```
${Date.now()}-${Math.random().toString(36).slice(2)}-${sanitized_original_name}
```

响应：

```
{ "success": true, "url": "/uploads/1704067200000-abc123-hotel.jpg" }
```

返回的 `url` 直接可用于 `<img src>` 或存入 Hotel.images JSON 数组。生产环境建议将 `public/uploads/` 替换为 OSS/CDN 存储，此接口仅需修改写入目标路径。

9.15 统一错误码与响应格式

系统通过 `app/lib/error-handler.ts` 统一处理所有 API 错误响应。核心函数 `handleApiError()`（`app/lib/error-handler.ts#L129-L168`）自动识别 Prisma 数据库错误、Zod 校验错误和通用异常，返回标准化 JSON 格式。`ERROR_MESSAGES` 常量表（`app/lib/error-handler.ts#L34-L55`）定义了 15+ 条用户友好的中文错误提示，避免向前端暴露技术细节。

成功响应：

```
{
  "success": true,
  "message": "操作成功提示",
  "data": { /* 业务数据 */ }
}
```

失败响应：

```
{
  "success": false,
  "message": "用户友好的错误描述"
}
```

HTTP 状态码对照表：

状态码	分类	含义	常见场景示例
200	成功	GET 请求成功返回数据	获取酒店列表、查询单个酒店详情
201	成功	POST 创建资源成功	创建预订、领取优惠券、提交评价
204	成功	DELETE 删除成功（无响应体）	删除收藏、取消预订
400	客户端错误	请求参数错误或业务规则校验失败	积分不足、优惠券已领取、房间已售罄、日期格式错误
401	客户端错误	未登录或 Token 失效	Token 已过期，请重新登录
403	客户端错误	无权限执行操作	当前角色无 HOTEL_AUDIT 权限
404	客户端错误	资源不存在	酒店 ID 不存在、预订记录未找到
409	客户端错误	资源冲突	该邮箱已注册（Prisma P2002 unique 冲突）
500	服务端错误	服务器内部错误	数据库连接失败、未捕获的异常
503	服务端错误	服务暂不可用	AI 推荐接口超时、第三方地图服务故障

业务错误码（message 字段常见值）：

错误信息	触发场景	对应文件
积分不足	用户积分 < 优惠券所需积分	`app/api/coupons/[id]/claim/route.ts#L44-L46`
优惠券已领取	UserCoupon 表复合主键冲突（P2002）	`app/api/coupons/[id]/claim/route.ts#L73-L75`

错误信息	触发场景	对应文件
优惠券已过期	当前日期不在 validFrom validTo 范围	`app/api/coupons/[id]/claim/route.ts#L31-L33`
房间已售罄	RoomAvailability.booked >= quota	`app/api/bookings/route.tsx#L208`
房间已关闭	RoomAvailability.isClosed = true	`app/api/bookings/route.tsx#L208`
邮箱已注册	User.email 唯一约束冲突（P2002）	`app/api/auth/register/route.tsx`
密码错误	bcrypt.compare 返回 false	`app/api/auth/login/route.tsx#L127`
用户不存在	prisma.user.findUnique 返回 null	`app/api/auth/login/route.tsx`
Token 已过期	jwt.verify 抛出 TokenExpiredError	`app/api/utils/auth.ts#L25`
无权限	Permission 表查询结果为空	`app/api/utils/permissions.ts#L110-L142`

错误处理工具链：API route 文件的 `catch` 块通过一行 `handleApiError(error)` 调用自动适配上述全部错误类型（`app/lib/error-handler.ts#L129`）——Prisma P2002 自动映射为“已存在”提示（`app/lib/error-handler.ts#L64-L73`），Zod 校验失败自动收集全部字段错误（`app/lib/error-handler.ts#L92-L108`），未知异常返回通用 500 错误。

10 部署与运行

10.1 环境依赖

依赖	推荐版本	说明
Node.js	≥ 20.x LTS	Next.js 16 要求 Node 18+，推荐 20 LTS 以获得最佳兼容性
npm	≥ 10.x	随 Node.js 附带安装
MySQL	8.x	本地安装或 Docker；需提前创建数据库 `hotel_management`
微信开发者工具	≥ 1.06.x	调试与预览小程序；需使用微信开发者账号登录
OpenAI 兼容接口	—	AI 推荐功能必须；支持 DouBao（火山引擎）、Kimi（月之暗面）、OpenAI 官方等兼容 `/v1/chat/completions` 的接口

10.2 环境变量配置

在项目根目录创建 `.env` 文件（此文件包含密钥，严禁提交至版本控制，已在 `.gitignore` 中排除）：

模板参考：`.env.example` · 加载位置：`app/lib/openai.ts` · `app/api/utils/auth.ts`

```
# ── 数据库连接字符串 ──
DATABASE_URL="mysql://root:yourpassword@localhost:3306/hotel_management"

# ── JWT 签名密钥（建议使用 32 位以上随机字符串，生产环境必须更换）──
JWT_SECRET="your-super-secret-jwt-key-here"

# ── AI 大模型配置（以火山引擎 DouBao 为例）──
OPENAI_API_KEY="your-api-key"
BASE_URL="https://ark.cn-beijing.volces.com/api/v3"
MODEL_ID="ep-20241231xxxxxx-xxxxx"

# 若使用 OpenAI 官方接口，BASE_URL 可省略，MODEL_ID 改为 "gpt-4o" 等
```

10.3 种子数据与测试账号

执行 `npm run db:seed` 后，系统自动写入以下基础数据（幂等，可重复执行）：

角色与权限：创建 USER / MERCHANT / ADMIN 三个角色，写入 `HOTEL_AUDIT`、`HOTEL_DELETE` 等细粒度权限，并将所有权限分配给 ADMIN 角色。

城市数据：预置 20 个热门城市（北京、上海、广州、深圳、杭州、成都等）。

标签数据：预置 20 个酒店标签（含早餐、免费 WiFi、游泳池、靠近地铁、温泉酒店等）。

示例酒店：执行 `prisma/seed-hotels.ts` 可额外写入多家含房型、图片、标签的示例酒店，供界面预览使用。

测试账号创建：种子脚本本身不写入用户数据，需通过注册接口或直接 SQL 插入创建测试账号。以下为推荐的测试账号结构：

角色	示例邮箱	示例密码	说明
ADMIN	`admin\@hotel.com`	admin123	注册后需在数据库将 roleId 改为 3（ADMIN）
MERCHANT	`merchant\@hotel.com`	merchant123	注册时选择“商户”角色，roleId=2

角色	示例邮箱	示例密码	说明
USER	user\@hotel.com	user123	默认注册即为普通用户，roleId=1

注意：bcryptjs 哈希密码需通过注册接口写入，不可明文存储。如需批量创建测试账号，可在 prisma/seed.ts 中追加 prisma.user.create 代码，密码使用 bcryptjs.hashSync('明文密码', 10) 预哈希。

10.4 后端 + Web 启动

配置文件：package.json · prisma/schema.prisma · .env.example

```
# 1. 安装项目依赖
npm install

# 2. 同步数据库结构至 MySQL
#    首次运行或 schema.prisma 变更后必须执行
npx prisma db push

# 3. 生成 Prisma Client 类型文件
#    db push 通常自动触发，可手动补充执行
npx prisma generate

# 4. 写入种子数据（角色/权限/城市/标签/示例酒店，可选但推荐）
npm run db:seed

# 5. 启动开发服务器（含热更新）
npm run dev
# 访问地址:
#   首页            → http://localhost:3000
#   管理员后台      → http://localhost:3000/admin/review
#   商户后台        → http://localhost:3000/merchant/dashboard
#   API 文档（Swagger）  → http://localhost:3000/api-doc

# ── 生产构建与启动 ──
npm run build
npm run start
```

10.5 小程序端启动

配置文件：trip_front_end_taro/package.json · trip_front_end_taro/src/app.config.js

```
# 1. 进入小程序子项目目录
cd trip_front_end_taro

# 2. 安装依赖
npm install

# 3. 启动微信小程序编译（watch 模式，保存后自动重编译）
npm run dev:weapp
```

```
# 编译产物位于 trip_front_end_taro/dist/weapp/

# 4. 在微信开发者工具中导入项目：
#    文件 → 导入项目 → 选择 dist/weapp 目录
#    AppID 填写自己的小程序 AppID 或选择"测试号"

# ── 其他平台 ──
npm run dev:h5          # H5 浏览器预览
npm run dev:alipay      # 支付宝小程序
npm run build:weapp     # 生产构建（微信），输出到 dist/weapp/
```

注意：小程序端 API 基础地址在 `trip_front_end_taro/src/services/request.js` 顶部配置，开发环境默认 `http://localhost:3000/api`。真机调试时需改为局域网 IP（如 `http://192.168.1.x:3000/api`）或已备案的线上域名，同时在微信小程序管理后台的「开发设置 → request 合法域名」中添加对应域名。

10.6 常见问题排查

问题现象	排查与解决方案
`navigateBack:fail`	开发者工具直接打开非首页时的正常报错，从 TabBar 首页正常跳转或真机运行均不会出现，可忽略
`getLocation` 权限被拒	`app.config.js` 中已声明 `permission.scope.userLocation`；开发者工具需在右上角“详情”→“本地设置”中手动允许地理位置权限
数据库连接失败	确认 MySQL 服务已启动（`brew services start mysql` 或 `systemctl start mysql`）；检查 `.env` 中 DATABASE_URL 的用户名/密码/库名；执行 `npx prisma db push` 可直接验证连通性
AI 接口返回 500	检查 `.env` 中 OPENAI_API_KEY / BASE_URL / MODEL_ID 是否正确；国内网络访问 OpenAI 官方接口需配置代理或改用 DouBao/Kimi 等国内兼容接口
图片上传后 404	确认 `public/uploads/` 目录存在且对 Node 进程有写权限；Next.js 开发模式下 public 目录变更需重启服务；生产环境建议改为 OSS 持久化存储
小程序地图显示空白	开发者工具：「设置 → 项目设置 → 不校验合法域名」；真机需在小程序后台添加 request 合法域名，并确认腾讯地图 SDK 使用了正确的 key
Prisma 类型报错	执行 `npx prisma generate` 重新生成 Client；确认 `prisma/schema.prisma` 已保存；确认 `.env` DATABASE_URL 指向正确数据库；生成文件路径在 `app/generated/prisma/`
端口 3000 被占用	`npm run dev -- -p 3001` 指定其他端口；同步修改小程序端 `services/request.js` 中的 baseURL；修改微信小程序后台 request 合法域名
Token 过期后请求 401	Web 端 `request.ts` 的 401 拦截器自动清除 localStorage 并跳转登录页；小程序端 `request.js` 同理跳转 login 页；如需无感刷新，使用 `/auth/refresh` 接口在 401 时换取新 Token

11 用户体验与兼容性

11.1 视觉设计与布局（6 分）

Web 管理端

采用 Ant Design 6.x 企业级组件库，提供统一的视觉规范与交互反馈：

- **深色/浅色双主题**：`next-themes` 的 `ThemeProvider` 管理全局主题，`data-theme` 属性驱动 CSS 变量切换，Navbar 一键切换，无闪烁；
- **响应式侧边栏布局**：商户后台和管理后台使用 Ant Design `Layout`（Sider + Content），宽屏下侧边导航展开，支持折叠；
- **反馈一致性**：所有异步操作均有 `message.success` / `.error` 提示，表单校验使用 `Form.Item rules` 内联错误展示；
- **数据可视化**：商户看板使用 Recharts 渲染折线图（7 日收入趋势）、柱状图（7 日预订量）、饼图（房型分布），直观展示运营数据。

小程序端

- **统一色系**：主色蓝 `#1890ff`，辅助灰/橙配色，与 Web 管理端保持视觉一致性；
- **卡片风格**：圆角（8px）+ 阴影（`box-shadow: 0 2px 12px rgba(0,0,0,0.08)`），层次清晰；
- **骨架屏**：酒店卡片、详情页、订单卡片等场景使用 `Skeleton` 组件渲染内容占位 + shimmer 光扫动画，提升感知加载速度；
- **统一图标**：借鉴 Phosphor Icons 开源库的 SVG path data（`trip_front_end_taro/src/utils/icons.js#L108-L119` getIconUri 函数生成 data URI），封装 `Icon` 组件通过 data URI 渲染，40+ 处调用统一风格，深色模式经 `tokens` 自动适配；
- **加载与空态**：`LoadingSpinner` 居中旋转动画避免白屏；无数据时统一渲染 `EmptyState` 组件（图标 + 提示文字），杜绝裸空白页；
- **动效**：酒店详情页滚动渐变 Header（0-150px 线性透明度插值），滚动切换 Tab 高亮，交互流畅自然。

11.2 浏览器与平台兼容性（4 分）

Web 管理端兼容性

基于 Next.js App Router（SSR + Client Components）和 Ant Design，支持所有主流现代浏览器：

浏览器	支持	说明
Chrome ⩾ 90	✓	主要开发与测试环境
Firefox ⩾ 88	✓	标准 CSS Flexbox/Grid 完全支持
Safari ⩾ 14	✓	Next.js SSR 首屏 HTML 无 JS 依赖，Safari 兼容良好
Edge ⩾ 90 (Chromium)	✓	同 Chrome 内核

小程序端多平台兼容性

Taro 4 将同一套 React 代码编译至多个平台，无需额外适配工作：

平台	支持	编译命令
微信小程序	✓ 主要调试平台	`npm run dev:weapp`
支付宝小程序	✓	`npm run dev:alipay`
抖音/字节小程序	✓	`npm run dev:tt`
H5 浏览器	✓	`npm run dev:h5`

12 创新点详解

12.1 提升研发效率的新技术（5 分）

技术	版本	研发效率提升
Taro 4	4.1.11	一套 React 代码编译微信/支付宝/抖音/H5，节省约 50% 多端适配工作量，CSS 自动转 rpx，组件 API 统一
Prisma ORM	6.19.2	Schema 驱动自动生成全套 TypeScript 类型（Model/Client/Query），消除手写 SQL 和运行时类型转换，迁移文件自动生成
LangChain	1.2.18	封装 LLM 调用链，`withStructuredOutput` + Zod Schema 实现意图提取的结构化输出，无需手写 JSON 解析和 prompt 校验逻辑
Zod	4.3.6	Schema 同时用于 API 入参运行时校验和 TypeScript 类型推导，一处定义全链路类型安全，与 LangChain 结构化输出复用同一套 Schema
next-themes	—	单行代码接入深色模式（`<ThemeProvider>`），无需手写 CSS 变量切换逻辑或 localStorage 管理
Swagger / JSDoc	swagger-ui-react	API 接口通过 JSDoc 注释自动生成 OpenAPI 3.0 文档并在 `/api-doc` 实时预览，接口变更后文档零维护成本同步更新

12.2 自主 UX 创新功能（5 分）

以下功能均为项目自主设计实现，超出题目基本要求：

1. **AI 自然语言酒店推荐**：用户以口语描述需求（如“找北京带早餐的五星级酒店”），系统两阶段处理：① 用 LLM（temperature=0）提取结构化搜索条件（目的地/价格/关键词），② 结合实时库存数据以流式输出（temperature=0.7）逐字推荐（`app/api/ai/recommend/route.ts#L42`），降低传统筛选表单的操作成本。**技术成本**：GPT-4o API 调用费用约 $2.50 每百万 tokens，平均每次推荐成本 <$0.002。

2. **可拖拽悬浮 AI 聊天按钮**：AI 聊天入口以悬浮圆形按钮呈现，通过 `onTouchStart/Move/End` 三事件以 3px 位移阈值区分“点击”与“拖动”，拖动时以 `Math.min/max` 限制在屏幕边界内，未拖动的 `TouchEnd` 触发打开聊天窗口。

3. **滚动渐变吸顶 Header**：详情页 Header 初始全透明，随页面下滑 0–150px 线性插值至不透明，酒店名称同步淡入；100px 阈值处标题文字切换，视觉层次自然过渡。

4. **酒店列表地图视图切换**：列表页支持“列表模式 ↔ 地图模式”无缝切换，地图模式以价格气泡（`callout.content`）标注各酒店位置（`trip_front_end_taro/src/pages/hotelMap/index.jsx#L115-L130`），点击气泡展示酒店缩略卡片，可直接预订。

5. **优惠券积分兑换全链路系统**：优惠券分为免费领取（`points = 0`）和积分兑换（`points > 0`）两种模式。免费券直接写入 `UserCoupon` 表；积分券通过 `prisma.$transaction` 事务保证原子性（`app/api/coupons/[id]/claim/route.ts#L36-L60`）——事务内三步：① `tx.user.findUnique` 查询余额 → ② `tx.user.update({ points: { decrement } })` 原子扣减 → ③ `tx.userCoupon.create` 发放券。余额不足抛 `INSUFFICIENT_POINTS` 回滚整个事务。前端 `Promise.allSettled` 并发加载全量券和已领记录，`Set<claimedIds>` O(1) 去重判定；下单时在预订确认弹窗选择可用券自动抵扣，后端 `@@id([userId, couponId])` 复合主键幂等保护。

6. **搜索历史去重持久化**：首页关键词搜索历史最多保留 10 条，`unshift` 插入后 `filter` 过滤重复项（`trip_front_end_taro/src/pages/home/index.jsx#L198-L210`），`Taro.setStorageSync` 跨会话持久化；历史条目一键回填搜索框，热门城市快捷入口降低输入成本。
7. **住宿类型多 Tab 联动**：首页四种住宿类型 Tab（国内/海外/钟点房/民宿）驱动整个搜索体系：钟点房自动切换 Calendar 为单日模式，其余 Tab 为区间模式；搜索时将类型参数序列化传至列表页做精确过滤。
8. **商户数据可视化看板**：商户后台 Dashboard 内嵌 Recharts 三种图表（折线图/柱状图/饼图），分别展示 7 日收入趋势、预订量变化和房型分布，所有数据由已有接口聚合计算，无需额外统计服务。
9. **评价防重复提交机制**：数据库 `Review` 表以 `bookingId @unique` 约束保证一单一评；后端拦截重复提交返回 Prisma P2002 错误；只有 `completed` 状态的订单才显示"提交评价"入口，从 UI 和数据层双重防护。
10. **A-Z 拼音城市索引**：自建工具模块 `trip_front_end_taro/src/utils/pinyinMap.js`，维护 70+ 常用汉字首字→拼音大写字母映射表（`CHAR_TO_INITIAL`），`groupByInitial()` 将全部城市按拼音首字母 A-Z 分组排序；城市选择弹窗右侧渲染字母索引栏，点击字母通过 `ScrollView.scrollIntoView` 快速定位对应分组，实现通讯录式城市检索体验。首页与列表页共享同一套组件逻辑与样式。
11. **搜索条件互斥与自动刷新**：关键词搜索与城市筛选设计为互斥关系——输入关键词时自动清除已选城市（全局搜索），选择城市时自动清空搜索关键词（定向筛选）；搜索框清空后 300ms 防抖自动重新加载全部结果，无需手动按回车，降低操作步骤。两个页面通过 `Taro.setStorageSync` 同步选择状态，保证跨页面一致性。
12. **列表页 N+1 查询消除**：后端 `GET /api/hotels` 接口通过 Prisma `include: { roomTypes: { select: { price: true } } }` 一次性返回所有酒店及其全部房型价格（单次 SQL JOIN），前端在内存中计算每个酒店的 `Math.min(roomTypes.map(rt => rt.price))`，彻底消除早期"每个酒店单独请求最低价"的 N+1 查询问题。10 条列表数据的响应延迟从约 500ms 降至约 150ms，数据库查询次数从 11 次降至 1 次。
13. **详情页并行化加载**：详情页需加载酒店基本信息、房型库存、收藏状态、评价列表四类数据，早期串行请求总耗时约 800ms。通过 `Promise.all([getHotelById(), getRoomTypes(), checkFavorite(), getComments()])` 并行发起，总耗时降至约 250ms（取决于最慢单个请求），首屏可交互时间（TTI）缩短约 70%，用户感知速度显著提升。单个接口失败时通过 `.catch(() => null)` 降级处理，不影响其他数据展示。

13 代码质量说明

13.1 项目结构与存储设计（4 分）

前后端共享类型层

`app/types/index.ts` 统一定义全项目的 TypeScript Interface（`Hotel`、`Booking`、`User`、`RoomType`、`Coupon` 等），Web 前端所有 Service 函数和页面组件共用同一套类型，杜绝前后端字段命名不一致导致的运行时错误。Prisma 自动生成的 Client 类型与此共享类型通过字段名约定保持对齐。

API 按业务域分目录

```
app/api/
├── auth/        # 认证（login / register / refresh）
├── hotels/      # 酒店 CRUD + 详情
├── bookings/    # 预订管理
├── reviews/     # 评价
├── favorites/   # 收藏
├── coupons/     # 优惠券
├── users/       # 用户管理
├── room-types/  # 房型与库存
├── tags/        # 标签
├── locations/   # 城市
├── ai/          # AI 推荐
├── upload/      # 文件上传
└── utils/       # verifyAuth / permissions
```

每个 `route.tsx` 文件职责单一（通常 100–300 行），业务逻辑与权限检查分离（`verifyAuth()` 抽象至 utils）。

核心工具与共享层（代码重构成果）

项目在代码质量优化中抽象出三个核心工具模块，消除重复逻辑并提升可维护性：

- `app/constants/index.ts` — 统一常量定义，包含 API 端点（`API_ENDPOINTS`）、状态码（`HOTEL_STATUS`、`BOOKING_STATUS`）、角色名（`ROLES`）、存储键（`STORAGE_KEYS`）等。所有魔法字符串替换为语义化常量，修改 API 路径只需更新一处即可全局生效，减少字符串拼写错误；
- `app/lib/auth-storage.ts#L18-L101` — 认证状态统一存储管理，封装所有 localStorage 操作为类型安全的 API。核心方法：`getToken()`（`app/lib/auth-storage.ts#L18`）、`saveAuth(token, refreshToken, user)`（`app/lib/auth-storage.ts#L79`）批量写入认证三元组、`clearAuth()`（`app/lib/auth-storage.ts#L89`）一键清除、`isAuthenticated()`（`app/lib/auth-storage.ts#L100`）检查登录态。自动处理 SSR 环境判断（`typeof window !== 'undefined'`）、JSON 序列化、角色校验，被约 8 个文件共享；
- `app/lib/error-handler.ts#L129-L168` — 标准化错误处理工具，`handleApiError()` 函数（`app/lib/error-handler.ts#L129`）统一处理三类异常：Prisma 数据库错误

（PrismaClientKnownRequestError，app/lib/error-handler.ts#L60-L87 含 P2002/P2025 等 6 种错误码映射）、Zod 校验错误（ZodError，app/lib/error-handler.ts#L92-L108 自动收集全部字段错误）和通用异常。ERROR_MESSAGES 常量表（app/lib/error-handler.ts#L34-L55）含 15+ 条用户友好的中文错误提示，API route 文件通过一行调用即可返回规范化错误响应：

来源：app/lib/error-handler.ts#L129-L168

```
// 统一错误处理函数——一行代码替代各 API 文件的重复 catch 块
export function handleApiError(error: unknown, defaultMessage: string):
ErrorResponse {
  if (error instanceof PrismaClientKnownRequestError) return
handlePrismaError(error);
  if (error instanceof ZodError) return handleZodError(error);
  if (error instanceof Error) { /* 检查已知业务错误 */ }
  return { error: defaultMessage, status: 500 };
}
```

数据库最小冗余设计

14 个数据表遵循以下设计原则：

- 多对多关系使用独立中间表（HotelTag、UserCoupon）且以复合主键代替自增 ID；
- RoomAvailability 以 (roomTypeId, date) 复合唯一索引替代应用层去重；
- HotelAuditLog 追加写入（不更新），审计记录不可篡改；
- JSON 字段（Hotel.facilities、Hotel.images）存储弱结构化数组，避免过度范式化。

13.2 编码规范与 README（3 分）

- **ESLint 统一规范**：项目根目录 eslint.config.mjs 配置 TypeScript ESLint 规则集，CI 本地运行 npx eslint 一键检查；
- **TypeScript 全链路**：后端 Next.js API Routes、前端 Web 页面、Service 层全部使用 TypeScript，Zod 在 API 边界做运行时校验，Prisma Client 提供数据库操作类型安全。后端 API 路由中所有动态构建的查询条件均定义了具体 TypeScript 接口（如 HotelWhereInput、PriceFilter）；前端组件定义 GuestInfo、UploadResponse、RoomFormItem 等业务接口，消除 18 处 any 类型（后端 13 处 + 前端 5 处），确保编译期类型检查覆盖；
- **命名约定**：JS/TS 变量和函数使用 camelCase，React 组件使用 PascalCase，数据库列名使用 camelCase（Prisma 惯例），CSS class 使用 kebab-case；
- **Service 层代码清理**：小程序端 7 个 service 文件（trip_front_end_taro/src/services/auth.js、trip_front_end_taro/src/services/booking.js、trip_front_end_taro/src/services/hotel.js 等）共清理约 52 处冗余 console.log 语句，统一由 trip_front_end_taro/src/services/request.js#L15-L25 的请求拦截器集中记录；清理 trip_front_end_taro/src/services/comments.js 中重复的 review 数据包装函数，单一职责原则下每个文件仅保留核心业务方法；

- **README 完备性**：项目根目录 `README.md` 包含：项目简介、技术栈、本地安装步骤、环境变量说明、数据库初始化命令（`prisma db push` + `db:seed`）、测试账号列表、常见问题排查，新成员可在 10 分钟内完成本地启动；
- **错误边界机制**：Next.js App Router 的 `app/error.tsx` / `app/loading.tsx` 和 `app/merchant/error.tsx` / `app/merchant/loading.tsx` 在全局和商户端实现统一错误处理和加载状态，防止白屏，改善用户体验；错误页面使用 Ant Design `Result` 组件展示友好提示，提供"重试"和"返回"按钮，加载状态使用 `Spin` 组件（已升级 `description` 属性替代废弃的 `tip`）。

13.3 代码复用与组件抽象（3 分）

可复用 UI 组件

组件	复用场景	说明
`Calendar`	首页日期选择、详情页日期修改	支持区间/单日双模式，底部固定确认按钮，Flex 可滚动布局保证按钮始终可见
`Icon`	全部页面的图标渲染（40+ 处）	借鉴 Phosphor Icons 开源图标库的 SVG path data，封装为 Taro 小程序兼容的 `<Image>` data URI 方案；支持 `name`/`size`/`color` 属性，`color` 接收 `tokens['--color-xxx']` 实现深色模式适配
`Skeleton`	酒店卡片、详情页、订单卡片加载态	支持 `hotelCard` / `hotelDetail` / `orderCard` 多种骨架屏类型，shimmer 动画
`AiChatWidget`	Web 端多页面、小程序端多页面	Web 版用 SSE 流式渲染，小程序版降级单次请求，同一组件接口
`FilterPanel`	酒店列表筛选、首页筛选	受控组件，接收 `defaultFilters` 初始值，`onConfirm` 回调
`LoadingSpinner`	所有数据加载场景	统一加载动画样式，替代各页面散装 loading 逻辑
`EmptyState`	列表无数据、搜索无结果	统一空状态 UI，含图标和自定义提示文字
`BookingConfirm`	详情页预订确认	含表单校验，与 `createBooking` 解耦
`SearchSuggestion`	首页搜索建议	展示搜索历史（最多 10 条，去重）和热门城市快捷入口；历史记录通过 `Taro Storage` 持久化，`unshift` 新搜索词并过滤重复项
`TencentMapSelector`	商户酒店录入页	Web 端腾讯地图坐标拾取组件，集成地图 SDK，支持地址搜索、标记拖拽、坐标回填表单
`Navbar`	Web 端全局导航	含主题切换按钮（调用 `useTheme().setTheme()`）、登录/注册入口、用户头像下拉菜单
`Footer`	Web 端页脚	展示版权信息、友情链接、ICP 备案号
`FeatureItem`	Web 端首页特性卡片	首页"系统特点"区块的卡片组件，接收 `icon`、`title`、`description` props
`HotelDetailsDrawer`	管理员酒店审核页详情展示	酒店详情抽屉组件，展示酒店基本信息、设施、图片、房型列表；通过 Ant Design Drawer + Descriptions 布局，图片支持预览组；从 `app/admin/hotels/page.tsx` 抽离（455 行优化至 222 行）
`HotelCommentsDrawer`	管理员酒店审核页评论管理	酒店评论抽屉组件，展示评论列表和删除操作；含 Popconfirm 二次确认、乐观 UI 更新（删除后立即从列表移除）
`useHotelTableColumns`	管理员酒店审核页表格列定义	自定义 Hook，集中管理表格列配置和操作按钮（查看、评论、下线、驳回、上线）；接收回调函数（`onView`、`onViewComments`、`onStatusChange`），返回 Ant Design Table

组件	复用场景	说明
		的 `columns` 配置；实现状态驱动的按钮显示逻辑（published 时显示下线/驳回按钮，offline 时显示上线按钮）

共享逻辑层

- `verifyAuth(request)` — 封装 JWT 解析 + Role 查询，被 29 个 API route 文件共享，修改一处即全局生效；
- `app/types/index.ts` — 统一 TypeScript 类型定义，Web Service 层和页面组件 import 同一来源；
- `trip_front_end_taro/src/utils/storage.js` — 小程序端统一封装 `getStorageSync/setStorageSync`，各页面通过 `storage.getToken()` / `storage.isAuthenticated()` 访问认证状态；
- `trip_front_end_taro/src/utils/useTheme.js` — 全局主题 Hook，各页面通过 `const { cssVars, isDark, tokens } = useTheme()` 一行获取 CSS 变量字符串、布尔标记及已解析色值对象，切换主题无需修改各页面代码；
- `trip_front_end_taro/src/utils/icons.js` — 集中存放 20+ 个 Phosphor Icons SVG path data 及 data URI 生成函数，`Icon` 组件按名称引用，新增图标只需添加一条 path entry；
- `app/lib/status-utils.ts` — Web 端预订状态工具，提供 `getStatusColor()` / `getStatusText()` 统一映射（`BookingStatus` → Ant Design Tag 颜色/中文文本），消除商户酒店管理、预订管理、仪表盘 3 个页面的重复代码；
- `trip_front_end_taro/src/utils/status.js` — Taro 端订单状态工具，提供 `getStatusText()` / `getStatusColor(status, tokens)` 动态颜色映射（支持深色模式），消除订单列表、订单详情 2 个页面的重复代码；
- `app/constants/index.ts` — 统一常量定义（API 端点、状态码、角色名、存储键），所有 Service 层和页面组件 import 同一来源，魔法字符串零重复；
- `app/lib/auth-storage.ts` — 认证状态管理单例，封装 Token/User 的读写、角色校验、SSR 环境守卫，被约 8 个文件共享；
- `app/lib/error-handler.ts` — 错误处理工具，统一处理 Prisma 错误、Zod 错误和通用异常，提供用户友好的中文错误消息映射表；
- `app/api/utils/permissions.ts` — 权限检查工具，实现 15 分钟 TTL 缓存机制（`PermissionCache` 类），提供单权限检查（`checkPermission`）和批量检查（`checkAnyPermission`、`checkAllPermissions`）API，性能提升 95%+。

14 架构决策记录

本章记录项目关键技术选型的决策理由与权衡，便于评审和后续维护参考。

14.1 ADR-1：Next.js App Router 全栈架构

背景：需要同时支持 Web 管理端和 REST API，团队规模小（3 人），希望减少技术栈碎片化。

决策：采用 Next.js 14 App Router，前后端同仓库、同一个 `app/` 目录结构。API Routes 直接作为后端服务，无需独立 Express/Koa 服务。

权衡：放弃了前后端分离部署的灵活性，换取统一部署、类型共享（`app/types/index.ts`）和更低运维成本。对于学生项目规模（14 张表、 30 个 API），单体架构足够。

14.2 ADR-2：Prisma ORM 替代原生 SQL

背景：团队需要快速建表、类型安全的数据库操作，且频繁修改 Schema。

决策：使用 Prisma 6 + `prisma/schema.prisma` 声明式 Schema，`prisma db push` 同步数据库结构，Client 自动生成全套 TypeScript 类型。

权衡：牺牲复杂查询的灵活性（如窗口函数、递归 CTE），换取开发速度和类型安全。实际项目中所有查询均可通过 Prisma Client API 表达（含 `$transaction`、嵌套 `include`、聚合 `_count`），未遇到需要 raw SQL 的场景。

14.3 ADR-3：JWT 双 Token 认证机制

背景：小程序和 Web 端均需无状态认证，且用户体验要求登录态持久化。

决策：Access Token（1h 有效期）+ Refresh Token（7d 有效期）双 Token 方案。Access Token 过期后前端拦截器自动以 Refresh Token 换取新 Token（`app/lib/request.ts#L47-L145`），请求队列排队等待刷新完成后重发。

权衡：比 Session 方案实现复杂度更高（需处理并发刷新、队列机制），但天然支持多端登录和横向扩展（无需共享 Session 存储）。

14.4 ADR-4：Taro 跨端方案替代原生开发

背景：需支持微信小程序为主、兼顾其他平台可能性，团队熟悉 React。

决策：Taro 4 编译型方案，React JSX 编写，编译至微信/支付宝/H5 等多平台。

权衡：相比原生 WXML 性能略有损耗，但一套代码多端复用节省约 50% 工作量。Taro 对微信原生 API（`getLocation`、`openLocation`、`Map` 组件）的封装兼容性良好，未遇到需要降级为原生写法的场景。

14.5 ADR-5：LangChain + Zod 结构化输出

背景：AI 推荐功能需从用户自然语言中提取结构化搜索条件，再基于数据库结果生成推荐文案。

决策：LangChain `withStructuredOutput(zodSchema)` 强制 LLM 输出符合 Zod Schema 的 JSON（`app/api/ai/recommend/route.ts#L42`），第一阶段 temperature=0 确保提取准确性，第二阶段 temperature=0.7 兼顾创意。

权衡：依赖 OpenAI API（单次调用成本约 $0.002），但两阶段分离保证了意图提取的确定性和推荐文案的多样性。Zod Schema 复用于 API 校验和 LLM 输出约束，减少重复定义。

15 安全设计总结

15.1 认证安全

- **密码存储**：bcrypt salt=10 哈希，数据库不存储明文密码（`app/api/auth/login/route.tsx#L3` `bcrypt.compare`）
- **JWT 密钥守护**：服务启动时强制校验 `JWT_SECRET` 非默认值，缺失或使用占位符直接抛出启动异常阻断服务（`app/api/utils/auth.ts#L4-L7`）
- **Token 双层机制**：Access Token 短期（1h）限制泄露窗口，Refresh Token 长期（7d）保持登录体验；前端拦截器自动刷新，用户无感知

15.2 接口安全

- **登录速率限制**：内存级 IP 频率限制器，同一 IP 15 分钟内最多 5 次登录尝试（`app/api/auth/login/route.tsx#L7-L24`），超限返回 429 + `Retry-After` 头
- **权限隔离**：`verifyAuth()`（`app/api/utils/auth.ts#L17-L30`）统一鉴权， 29 个 API route 共享；商户只能操作自有酒店，管理员通过 `checkPermission()`（`app/api/utils/permissions.ts#L128-L143`）细粒度控制。所有权限检查统一使用 `checkPermission()` 函数替代各 API 路由中重复的 `prisma.user.findUnique({ include: { role: { include: { rolePermission } } } })` 查询（`app/api/users/route.tsx`、`app/api/users/[id]/route.tsx`、`app/api/hotels/[id]/route.tsx`、`app/api/room-types/[id]/route.tsx` 共 5 处替换），复用 15 分钟 TTL 缓存避免重复数据库查询
- **Zod 边界校验**：所有 API 入参经 Zod Schema 运行时校验（如 `app/api/bookings/route.tsx#L7-L15`），非法数据在进入业务逻辑前即被拦截

15.3 数据安全

- **事务原子性**：涉及资金/积分操作使用 `prisma.$transaction` 保证一致性（预订扣库存、积分兑券），异常自动回滚
- **幂等保护**：复合主键（`UserCoupon @@id([userId, couponId])`）和唯一约束（`Review bookingId @unique`、`RoomAvailability @@unique([roomTypeId, date])`）从数据层防重复
- **审计追踪**：`HotelAuditLog` 表追加写入（不更新/删除），记录酒店状态变更操作者和时间

15.4 前端安全

- **Token 安全存储**：Web 端使用 `localStorage`（`app/lib/auth-storage.ts#L18`），小程序端使用 `Taro.getStorageSync`，均封装为统一 API 避免直接操作
- **请求拦截**：axios 请求拦截器自动注入 `Authorization` Header（`app/lib/request.ts#L32-L44`），响应拦截器处理 401 自动刷新 Token 或跳转登录
- **并发刷新控制**：`isRefreshing` 锁 + `failedQueue` 队列（`app/lib/request.ts#L12-L16`）确保多个并发 401 请求只触发一次 Token 刷新，其余排队等待

16 后端查询性能优化

16.1 API 响应字段精简

核心文件：`app/api/hotels/route.tsx`（酒店列表）、`app/api/favorites/route.tsx`（收藏列表）

做了什么：酒店列表和收藏列表接口原先使用 `include: { roomTypes: true }` 返回完整房型对象（含 description、amenities、images、stock、availability 等全部字段），移动端列表页仅需 `id` 和 `price` 两个字段。

怎么做的：将 `include` 改为 `select` 精确指定返回字段：

```
// 优化前: 返回完整 roomType 对象（~20 字段 × N 条房型）
include: { roomTypes: true }

// 优化后: 仅返回列表页所需字段
roomTypes: { select: { id: true, price: true } }
```

同样优化了 `location`（仅返回 `id/name/type`）、`merchant`（仅返回 `id/name`）、`hotelTags`（仅返回 `tag.id/tag.name`）的关联查询。

性能收益：单次 API 响应数据量减少约 40-50%，对移动端弱网环境尤为显著。

16.2 预订事务批量查询优化

核心文件：`app/api/bookings/route.tsx#L244-L330`

做了什么：预订接口创建订单时需要检查入住区间内每一天的房间可用性。原实现对每一天执行 3 次数据库操作（`upsert` + `findUnique` + `updateMany`），7 晚预订共 21 次数据库交互。

怎么做的：将查询逻辑重构为两阶段：

- **批量查询阶段**：`findMany` 一次性获取所有日期记录 → `createMany({ skipDuplicates })` 补齐缺失日期 → 再次 `findMany` 获取完整数据。共 3 次数据库操作替代原先 N×2 次查询。
- **逐日原子更新阶段**：仍使用 `updateMany({ where: { booked: { lt: quota } } })` 保证并发安全，但省去了每日的 `upsert` 和 `findUnique`。

性能收益：7 晚预订从 21 次降至 10 次数据库操作（减少 52%），预订接口响应时间降低约 30%。

16.3 类型安全加固

涉及文件：`app/api/hotels/route.tsx`、`app/api/hotels/[id]/route.tsx`、`app/api/bookings/route.tsx`、`app/api/bookings/[id]/route.tsx`、`app/api/users/[id]/route.tsx`、`app/api/tags/route.tsx`、`app/api/locations/route.tsx`、`app/api/comments/route.ts`、`app/api/auth/register/route.tsx`

做了什么：消除后端 API 路由中全部 13 处 `any` 类型注解，替换为具体的 TypeScript 接口或内联类型。

典型示例：

```
// 优化前
const where: any = {};
const priceFilter: any = {};

// 优化后
interface HotelWhereInput {
  locationId?: number;
  merchantId?: number;
  type?: string;
  starRating?: { gte: number };
  roomTypes?: RoomTypeFilter;
  OR?: Array<{ nameZh?: { contains: string } }>;
}
const where: HotelWhereInput = {};
const priceFilter: PriceFilter = {};
```

收益：编译器可在开发阶段捕获字段拼写错误和类型不匹配，减少运行时异常风险。